

Security at the Transport Layer: SSL and TLS

Introduction

- In online transaction, following security services are desired.
 - **Entity authentication**: The customer needs to be sure of the server belongs to the actual vender.
 - **Message integrity**: The customer and the vender need to be sure that the contents of the message are not modified during transmission.
 - **Confidentiality**: The customer and the vender need to be sure that an attacker does not intercept sensitive information.

Introduction

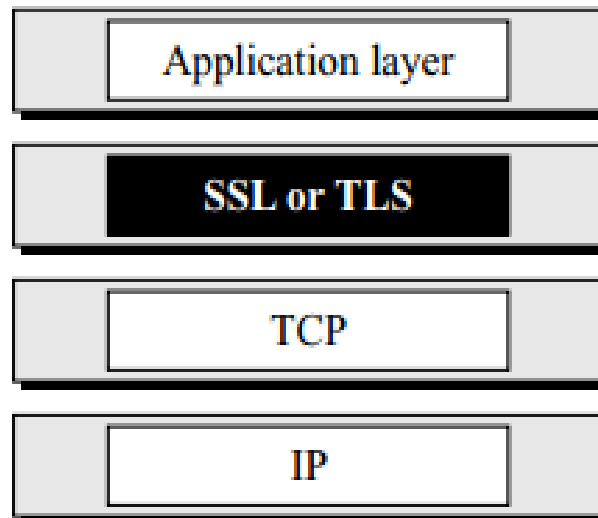
- Two protocols are dominant today for providing security at the transport layer:
 - Secure Sockets Layer (SSL) Protocol
 - Transport Layer Security (TLS) Protocol.
- They are both cryptographic protocols that provide secure communications on the Internet for such things as web browsing, e-mail, Internet faxing, instant messaging, and other data transfers.

Who Uses SSL?

- SSL is the de facto standard for encrypted and authenticated communications between clients and servers on the Internet.
 - *Financial institutions implement SSL to secure the transmission of PIN numbers and other confidential account information.*
 - *Insurance companies implement SSL to secure transmission of confidential policy information.*
 - *Private organizations implement SSL in their intranets to confidentially transfer information to and from employees.*

Location of SSL and TLS in the Internet model

- Application-layer client/server programs, such as Hypertext Transfer Protocol (HTTP), that use the services of TCP can encapsulate their data in SSL packets.



Services provided by SSL

- Fragmentation
 - SSL divides the data into blocks of 2^{14} bytes or less
- Compression
 - Each fragment of data is compressed using one of the lossless compression methods
 - Optional
- Message Integrity
 - SSL uses a keyed-hash function to create a MAC
- Confidentiality
 - original data and the MAC are encrypted using symmetric key cryptography.

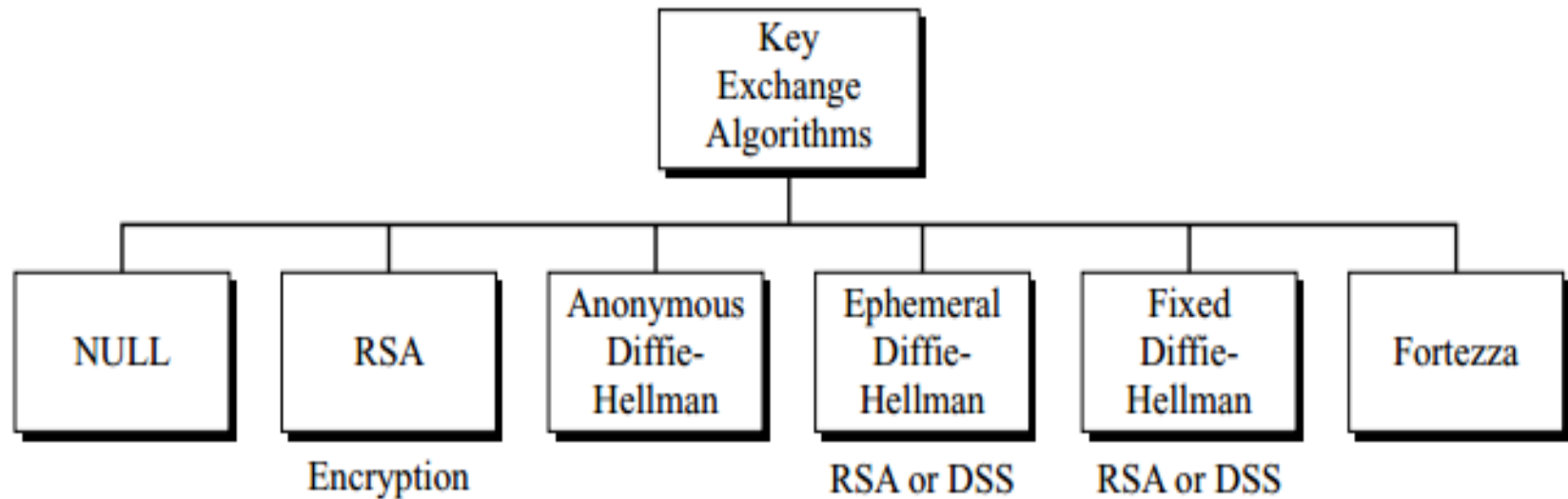
Services provided by SSL

- Framing
 - A header is added to the encrypted payload.
 - The payload is then passed to a reliable transport layer protocol.

Key-exchange methods of SSL

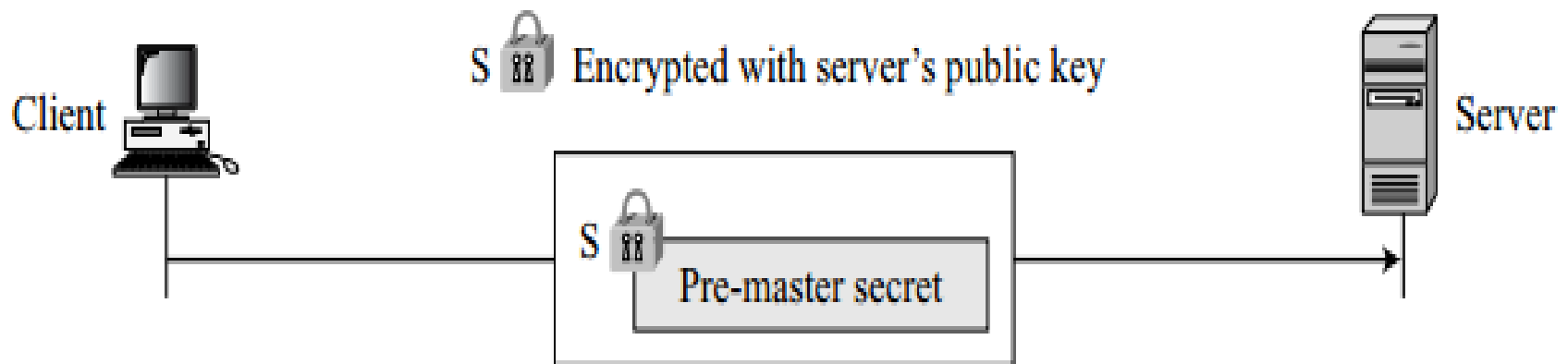
- To exchange an authenticated and confidential message, the client and the server each need **six** cryptographic secrets
 - four keys and two initialization vectors.
- To create these secrets, one **pre-master secret** must be established between the two parties.
 - Both client and server need to know the value of the pre-master secret.

Key-exchange methods of SSL



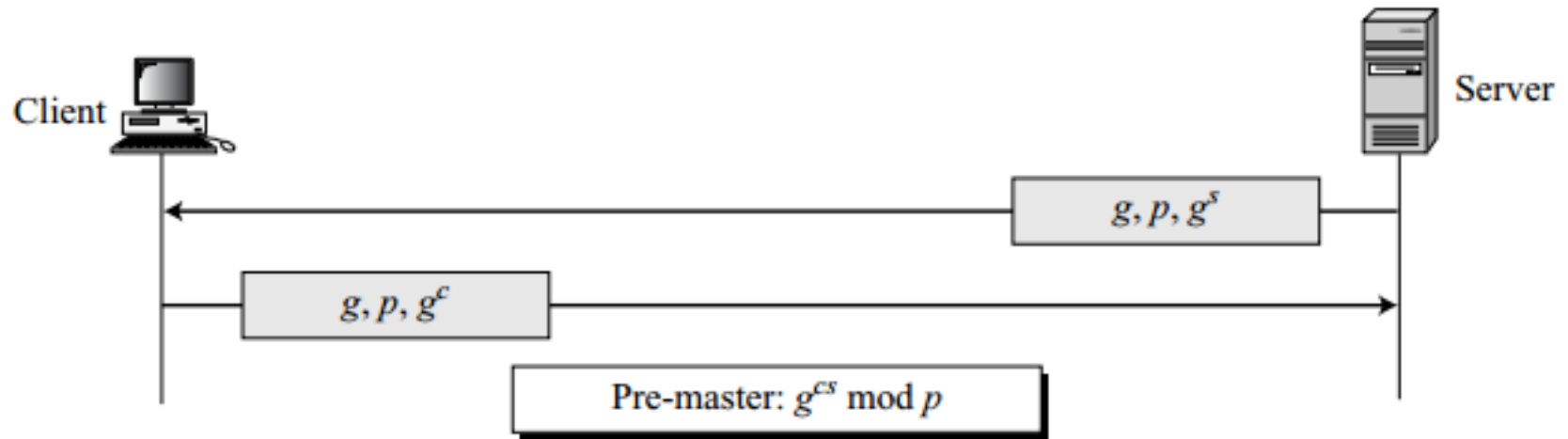
Key-exchange methods of SSL

- **RSA**
 - pre-master secret is a 48-byte random number created by the client, encrypted with the server's RSA public key, and sent to the server.



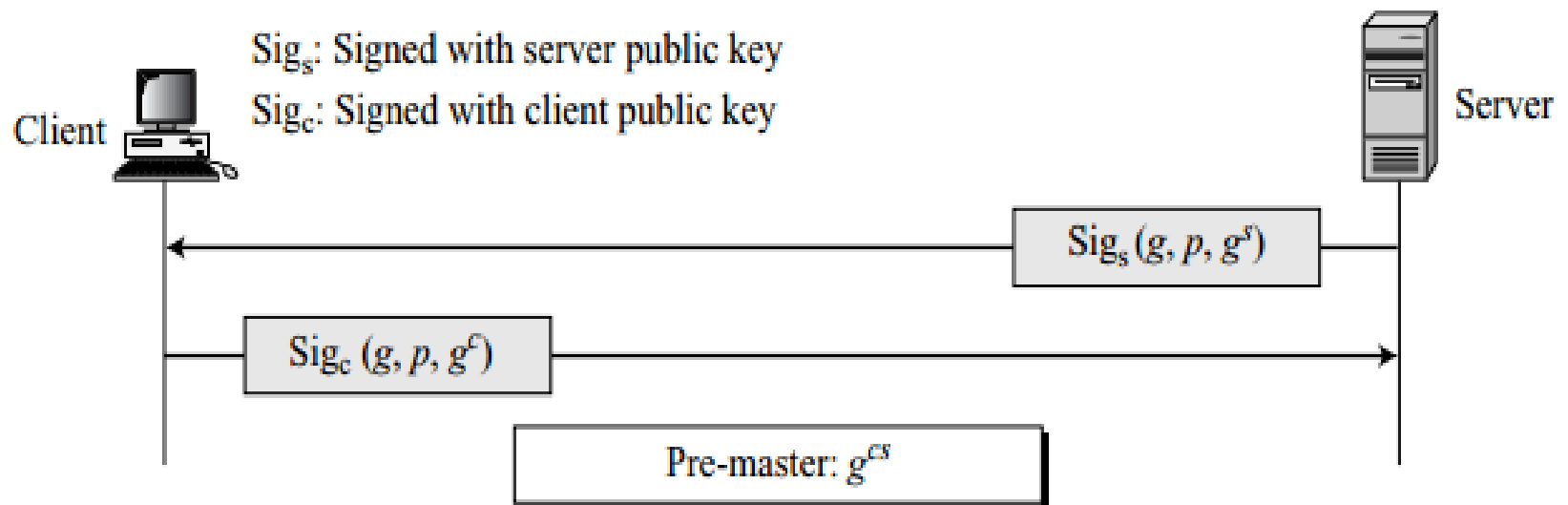
Key-exchange methods of SSL

- Anonymous Diffie-Hellman
 - simplest and insecure
 - The Diffie-Hellman **half-keys** are sent in plaintext
 - neither party is known to the other



Key-exchange methods of SSL

- **Ephemeral Diffie-Hellman**
 - Each party sends a Diffie-Hellman key signed by its private key.
 - The receiving party needs to verify the signature using the public key of the sender.



Key-exchange methods of SSL

- **Fixed Diffie-Hellman**

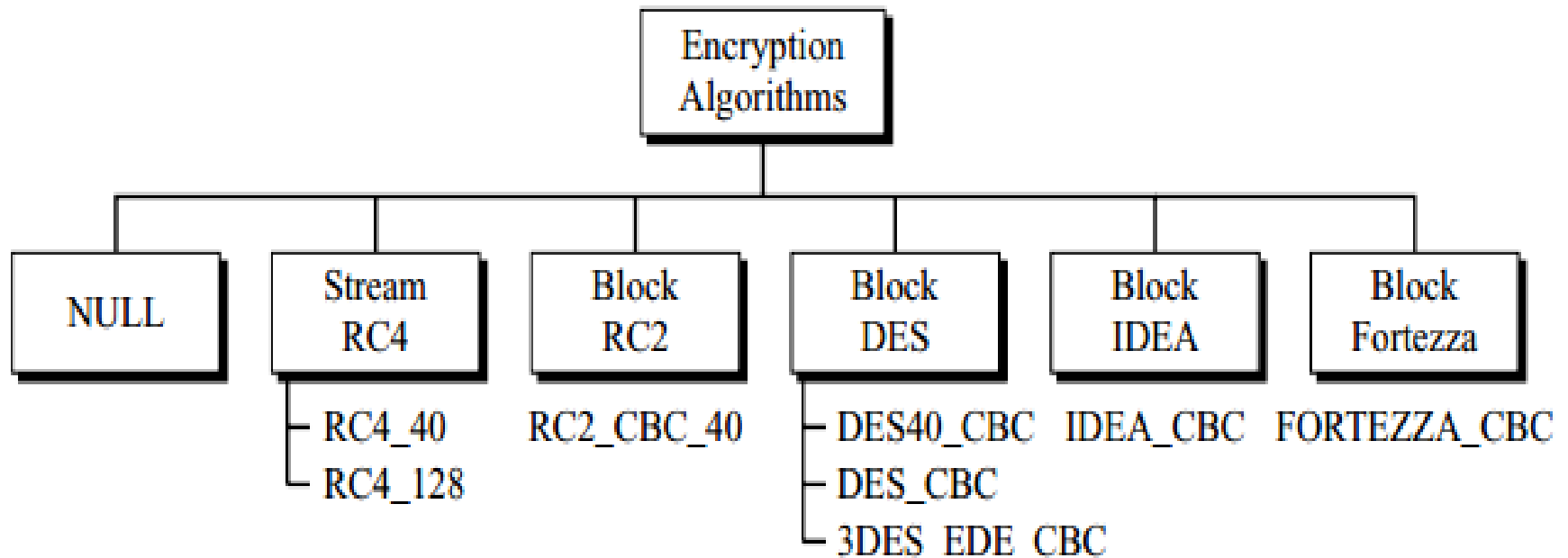
- All entities in a group can prepare fixed Diffie-Hellman parameters (g and p).
- each entity can create a fixed Diffie-Hellman half-key (g^x).
- each individual half-key is inserted into a certificate verified by a certification authority (CA)
- A client uses its own fixed half-key and the server half-key received in a certificate to compute his premaster key.

Key-exchange methods of SSL

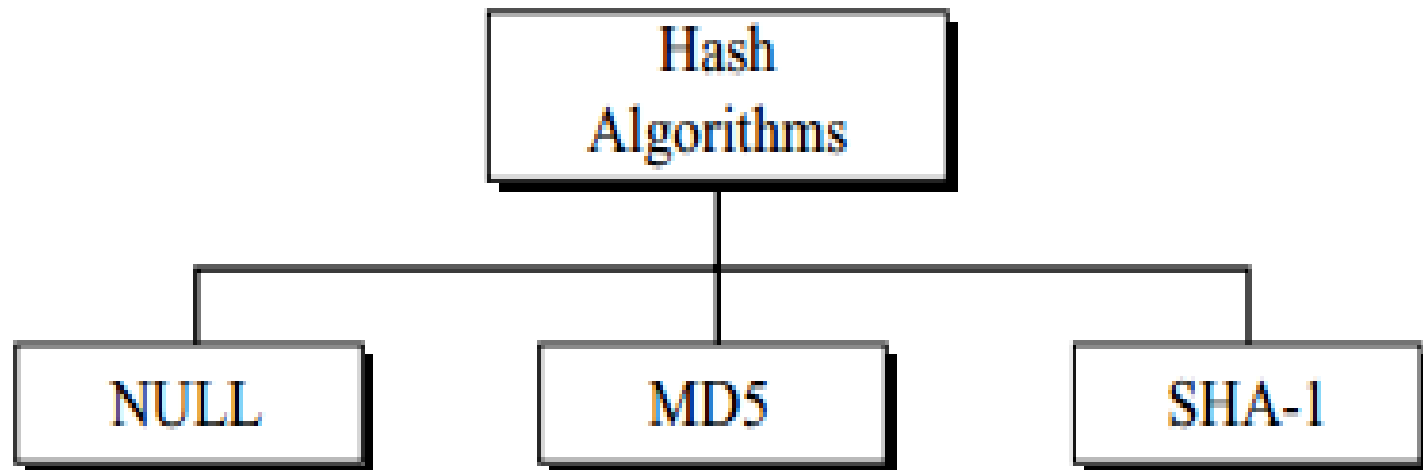
- Fortezza
 - It is a registered trademark of the U.S. National Security Agency (NSA).

Encryption/Decryption Algorithms

- All block protocols use an 8-byte initialization vector (IV) except for Fortezza, which uses a 20-byte IV.



Hash Algorithms



Cipher Suite of SSL

- The combination of key exchange, hash, and encryption algorithms defines a **cipher suite** for each SSL session.
- For example,

`SSL_DHE_RSA_WITH_DES_CBC_SHA`

SSL cipher suite list

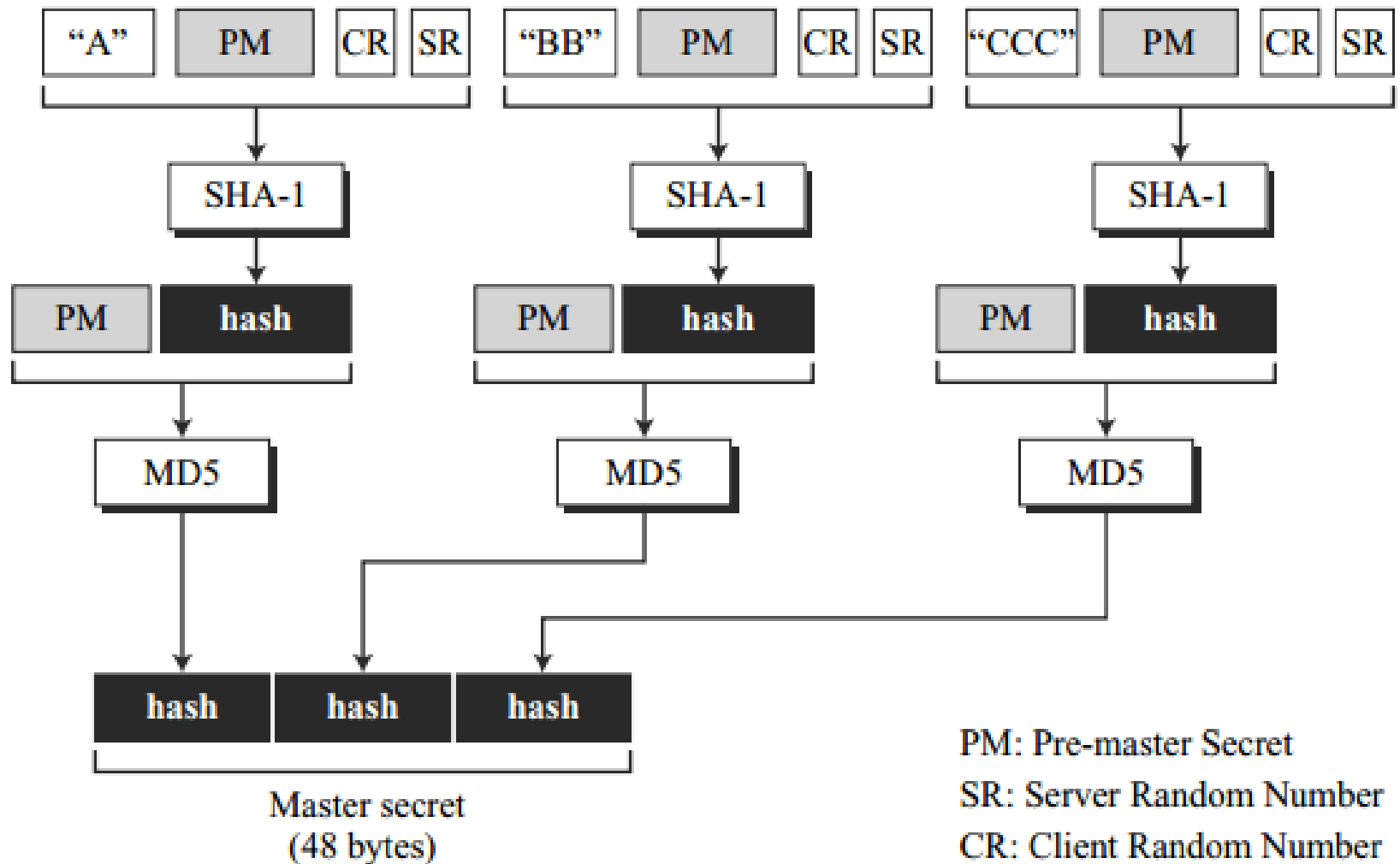
TABLE 1.1.1 SSL Cipher Suite List

<i>Cipher suite</i>	<i>Key Exchange</i>	<i>Encryption</i>	<i>Hash</i>
SSL_NULL_WITH_NULL_NULL	NULL	NULL	NULL
SSL_RSA_WITH_NULL_MD5	RSA	NULL	MD5
SSL_RSA_WITH_NULL_SHA	RSA	NULL	SHA-1
SSL_RSA_WITH_RC4_128_MD5	RSA	RC4	MD5
SSL_RSA_WITH_RC4_128_SHA	RSA	RC4	SHA-1
SSL_RSA_WITH_IDEA_CBC_SHA	RSA	IDEA	SHA-1
SSL_RSA_WITH_DES_CBC_SHA	RSA	DES	SHA-1
SSL_RSA_WITH_3DES_EDE_CBC_SHA	RSA	3DES	SHA-1
SSL_DH_anon_WITH_RC4_128_MD5	DH_anon	RC4	MD5
SSL_DH_anon_WITH_DES_CBC_SHA	DH_anon	DES	SHA-1
SSL_DH_anon_WITH_3DES_EDE_CBC_SHA	DH_anon	3DES	SHA-1
SSL_DHE_RSA_WITH_DES_CBC_SHA	DHE_RSA	DES	SHA-1
SSL_DHE_RSA_WITH_3DES_EDE_CBC_SHA	DHE_RSA	3DES	SHA-1
SSL_DHE_DSS_WITH_DES_CBC_SHA	DHE_DSS	DES	SHA-1
SSL_DHE_DSS_WITH_3DES_EDE_CBC_SHA	DHE_DSS	3DES	SHA-1
SSL_DH_RSA_WITH_DES_CBC_SHA	DH_RSA	DES	SHA-1
SSL_DH_RSA_WITH_3DES_EDE_CBC_SHA	DH_RSA	3DES	SHA-1
SSL_DH_DSS_WITH_DES_CBC_SHA	DH_DSS	DES	SHA-1
SSL_DH_DSS_WITH_3DES_EDE_CBC_SHA	DH_DSS	3DES	SHA-1
SSL_FORTEZZA_DMS_WITH_NULL_SHA	Fortezza	NULL	SHA-1
SSL_FORTEZZA_DMS_WITH_FORTEZZA_CBC_SHA	Fortezza	Fortezza	SHA-1
SSL_FORTEZZA_DMS_WITH_RC4_128_SHA	Fortezza	RC4	SHA-1

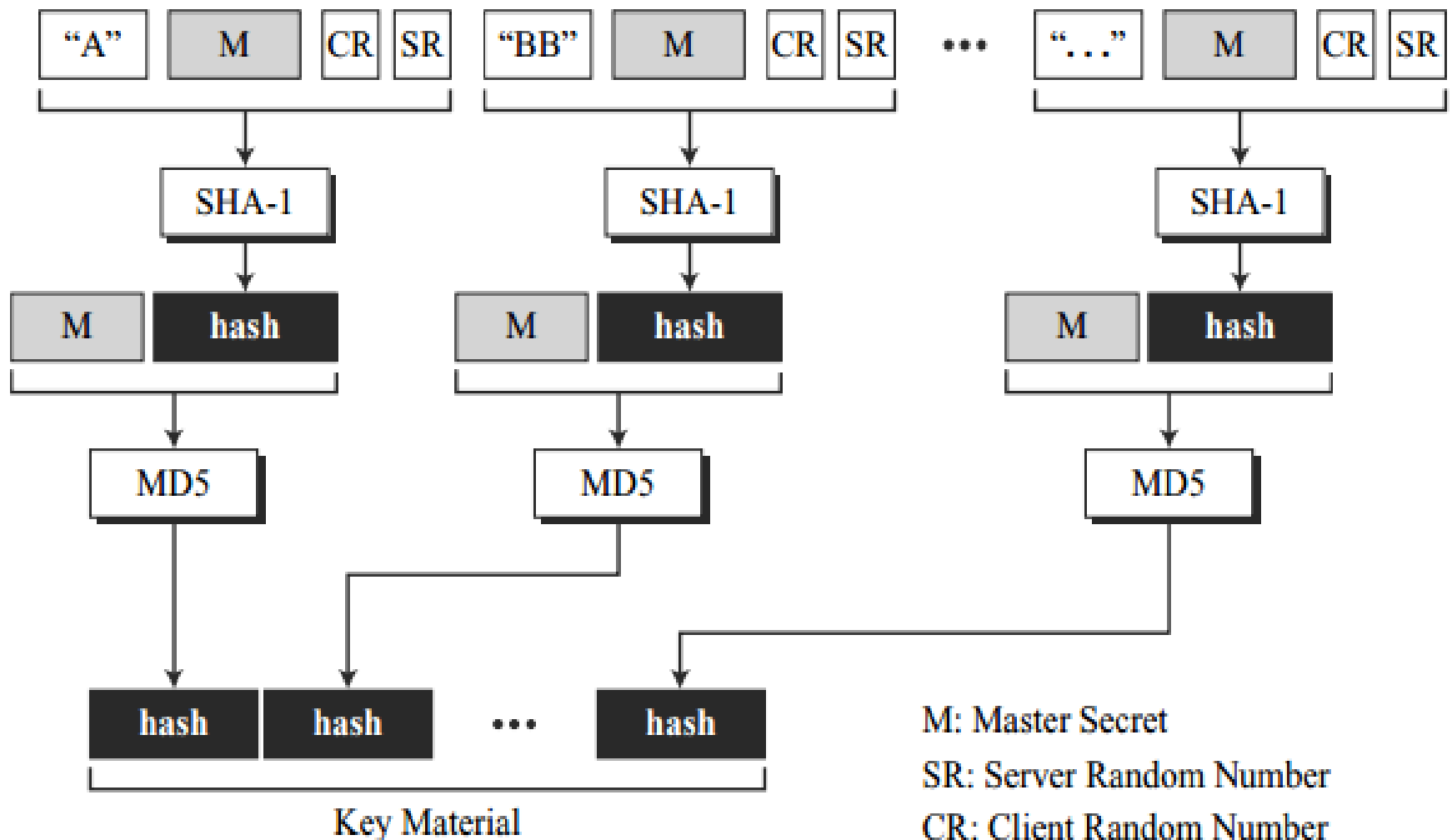
Cryptographic Parameter Generation

1. The client and server exchange two random numbers; one is created by the client and the other by the server.
2. The client and server exchange one pre-master secret using one of the key-exchange algorithms
3. A 48-byte **master secret** is created from the **pre-master secret** by applying two hash functions (SHA-1 and MD5)
4. Calculation of **key material** from **master secret**
5. Extractions of **cryptographic secrets** (Six different keys) from **key material**

Calculation of master secret from pre-master secret



Calculation of key material from master secret



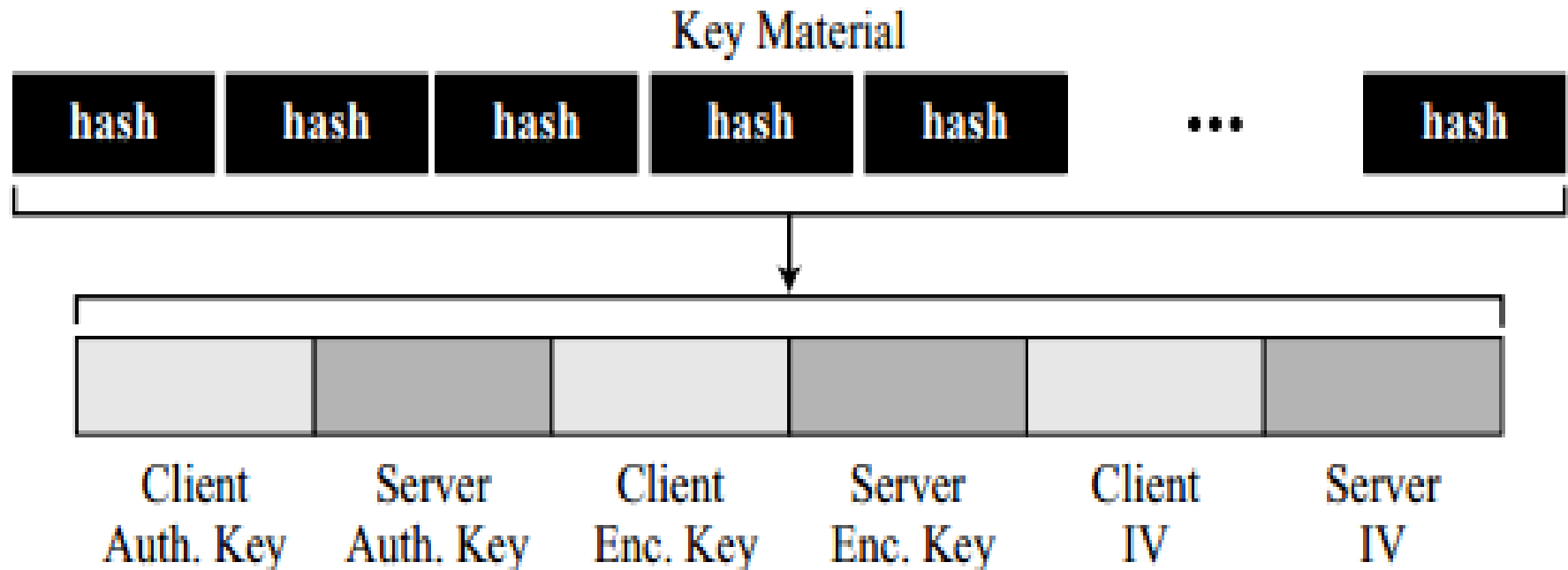
- The length of the key material block depends on the cipher suite selected and the size of keys needed for this suite.

Extractions of cryptographic secrets from key material

Auth. Key: Authentication Key

Enc. Key: Encryption Key

IV: Initialization Vector



Sessions and Connections in SSL

- A **session** is an association between a client and a server.
- After a session is established, the two parties have common information such as:
 - session identifier
 - certificate authenticating each of them (if necessary)
 - the compression method (if needed)
 - cipher suite
 - a master secret that is used to create keys for message authentication encryption

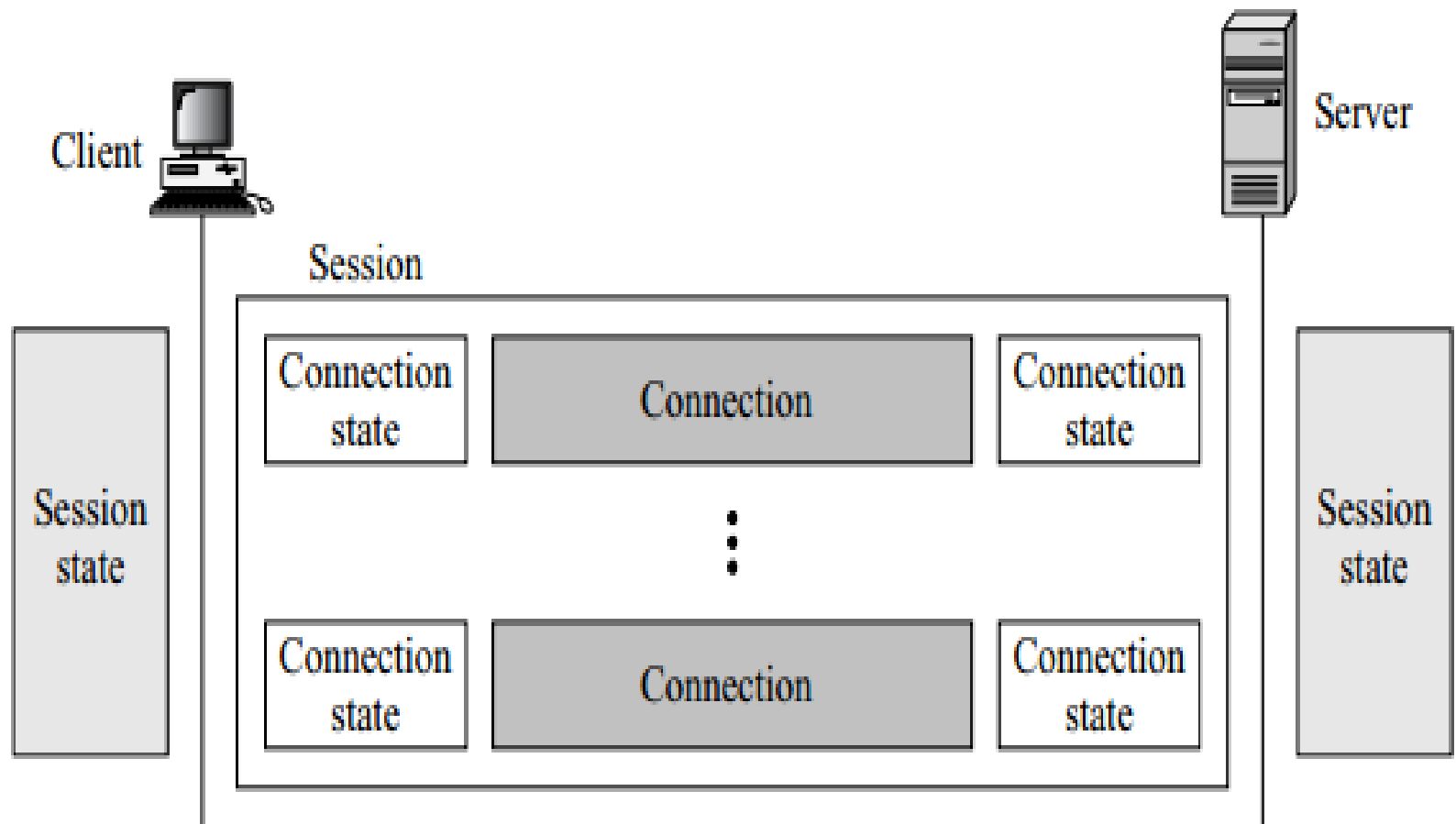
Sessions and Connections in SSL

- A **session** can consist of many connections.
- A connection between two parties can be terminated and reestablished within the same session.
- When a connection is terminated, the two parties can also terminate the session, **but it is not mandatory**.
 - A session can be suspended and resumed again.

Sessions and Connections in SSL

- In a session, one party has the role of a client and the other the role of a server
- In a connection, both parties have equal roles, they are peers.

Sessions and Connections in SSL



- **Session State:**
 - A session is defined by a session state, a set of parameters established between the server and the client.
- Session state parameters

<i>Parameter</i>	<i>Description</i>
Session ID	A server-chosen 8-bit number defining a session.
Peer Certificate	A certificate of type X509.v3. This parameter may be empty (null).
Compression Method	The compression method.
Cipher Suite	The agreed-upon cipher suite.
Master Secret	The 48-byte secret.
Is resumable	A yes-no flag that allows new connections in an old session.

- Connection State

- A connection is defined by a connection state, a set of parameters established between two peers.
- SSL uses two attributes to distinguish cryptographic secrets: **write** and **read**.
- **write** specifies the key used for signing or encrypting outbound messages.
- The term **read** specifies the key used for verifying or decrypting inbound messages.

- The **write key** of the client is the same as the read key of the server
- The **read key** of the client is the same as the write key of the server.

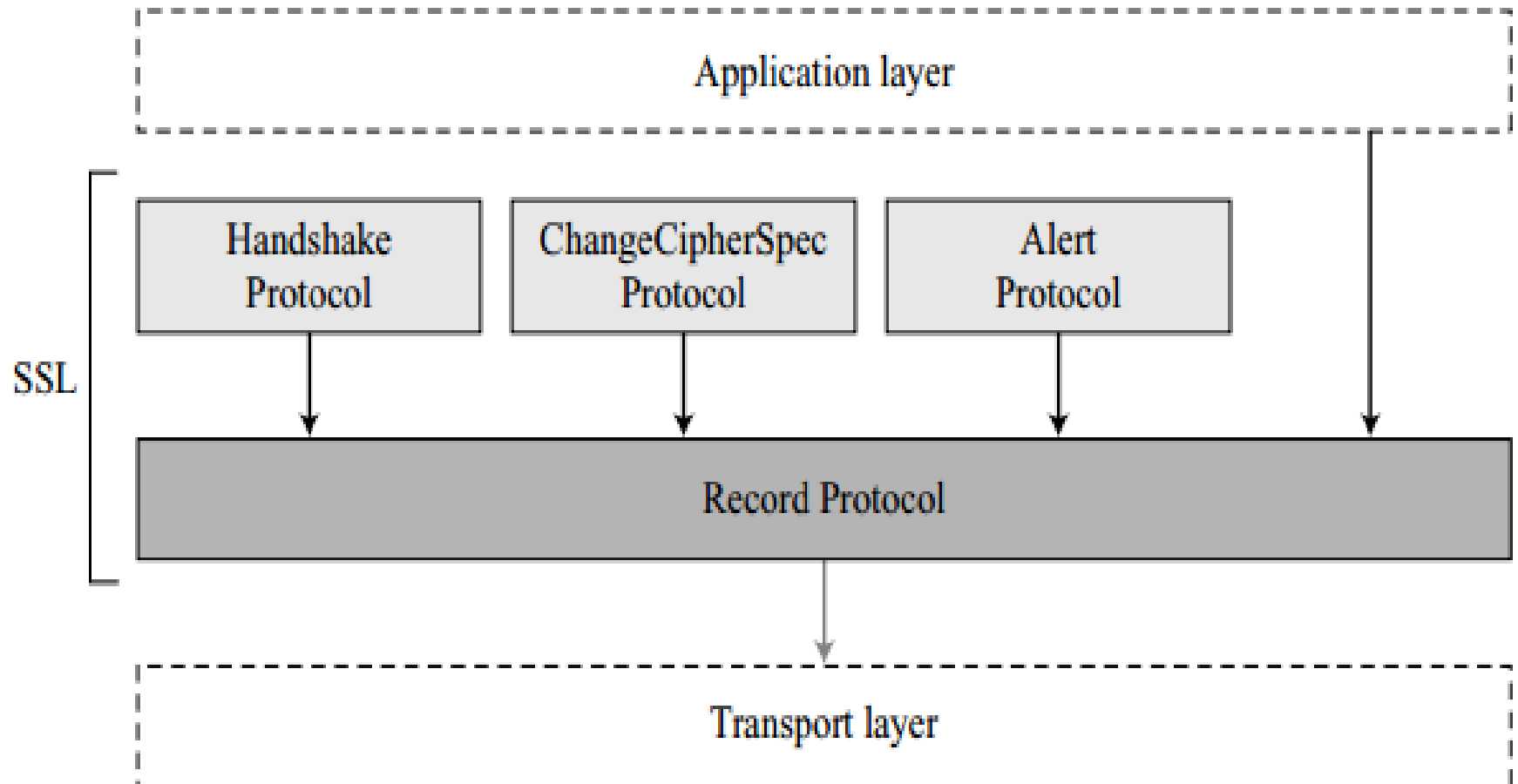
Connection state parameters

<i>Parameter</i>	<i>Description</i>
Server and client random numbers	A sequence of bytes chosen by the server and client for each connection.
Server write MAC secret	The outbound server MAC key for message integrity. The server uses it to sign; the client uses it to verify.
Client write MAC secret	The outbound client MAC key for message integrity. The client uses it to sign; the server uses it to verify.
Server write secret	The outbound server encryption key for message integrity.
Client write secret	The outbound client encryption key for message integrity.
Initialization vectors	The block ciphers in CBC mode use initialization vectors (IVs). One initialization vector is defined for each cipher key during the negotiation, which is used for the first block exchange. The final cipher text from a block is used as the IV for the next block.
Sequence numbers	Each party has a sequence number. The sequence number starts from 0 and increments. It must not exceed $2^{64} - 1$.

SSL Protocols

- Four SSL protocols:
 - Handshake Protocol
 - Record Protocol
 - ChangeCipherSpec Protocol
 - Alert Protocol

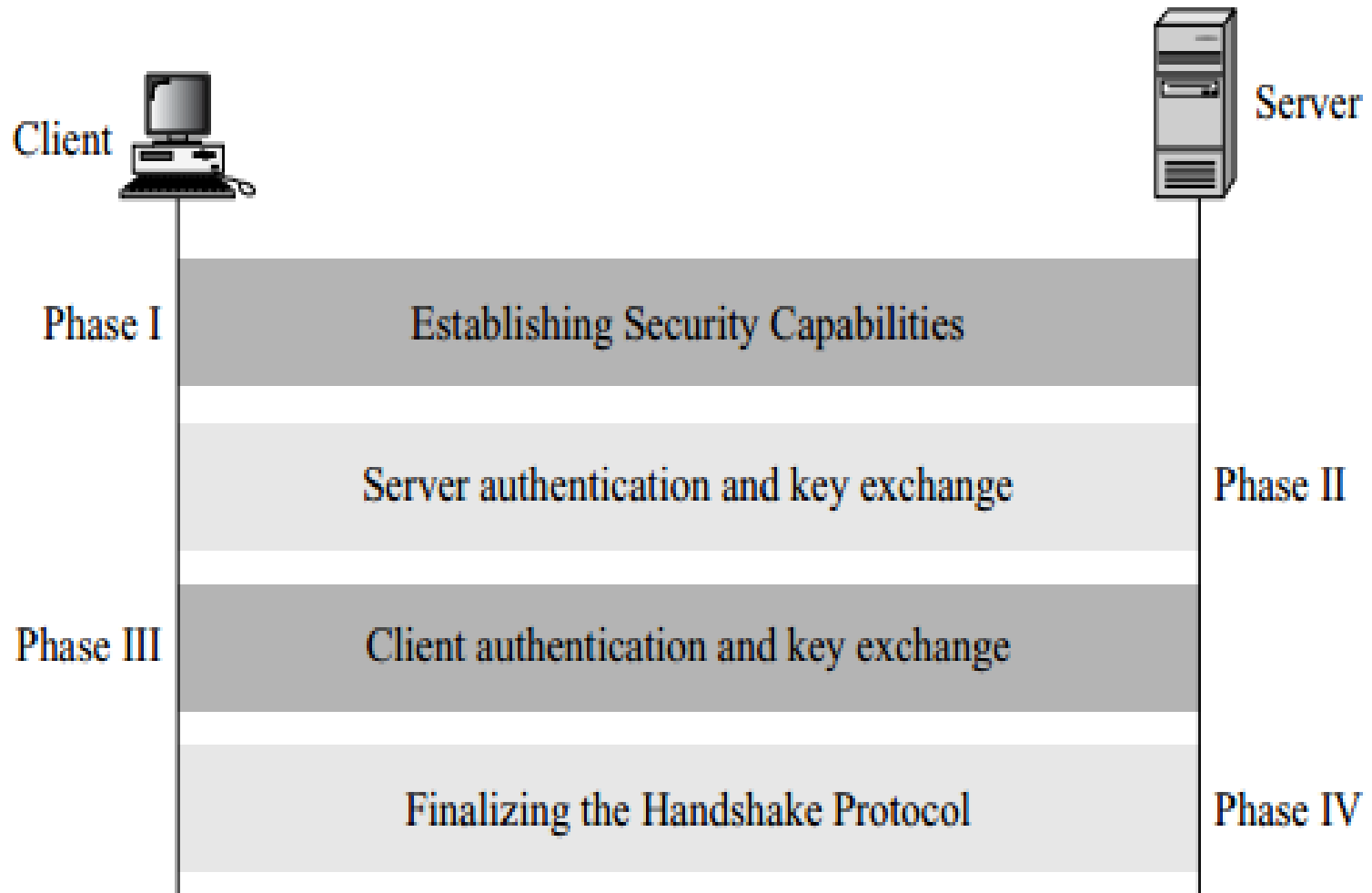
SSL Protocols



Handshake Protocol

- It uses messages to
 - Negotiate the cipher suite
 - Authenticate the server to the client and vice versa (if needed)
 - Exchange information for building the cryptographic secrets
- The handshaking is done in **four phases**.

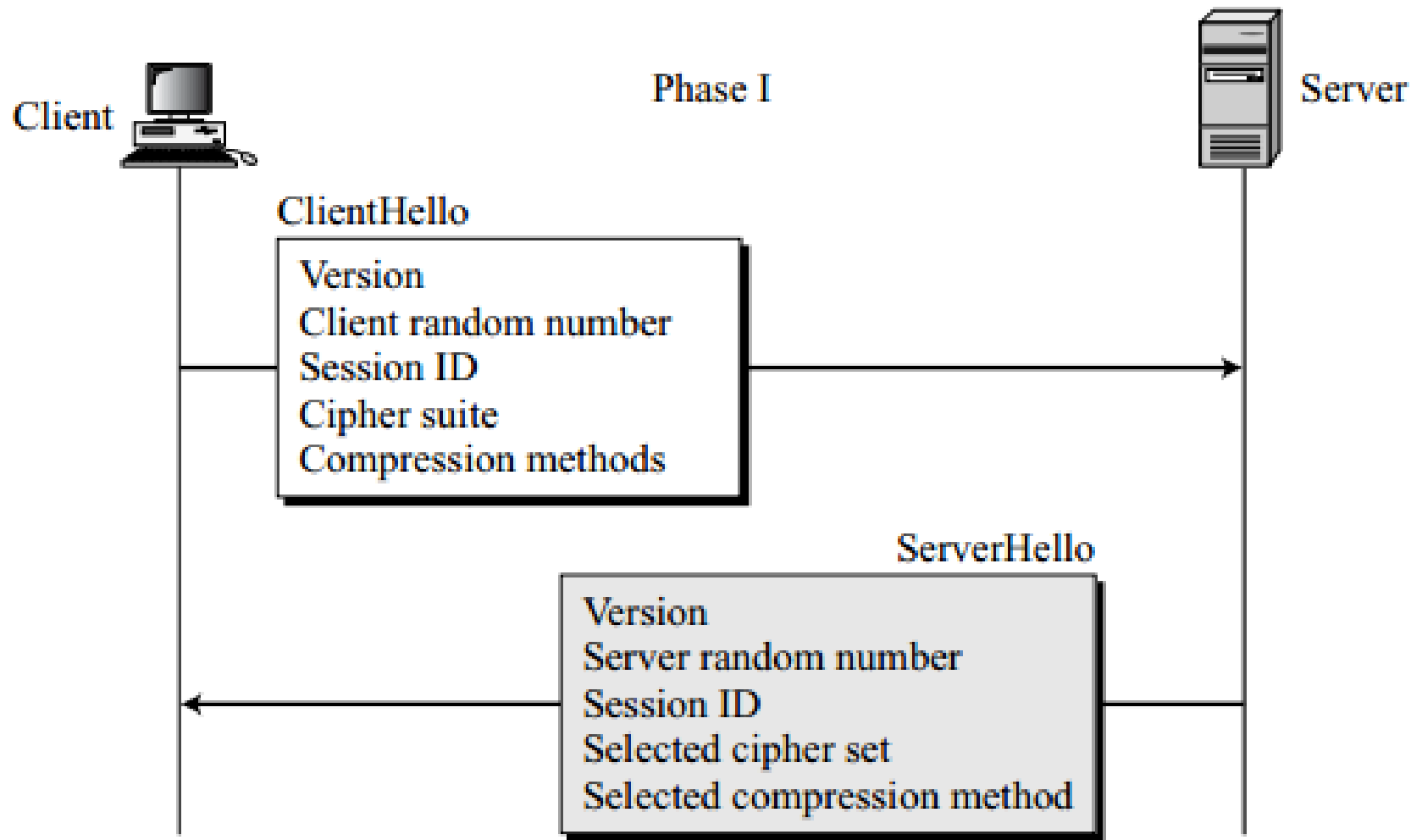
Four Phases of Handshake Protocol



Phase I: Establishing Security Capability

- Client and server announce their security capabilities and choose those are convenient for both.
 - a session ID is established
 - cipher suite is chosen
 - compression method (**optional**) is chosen
 - Client-random and server-random numbers are created
- Two messages are exchanged in this phase:
 - **ClientHello**
 - **ServerHello**

Phase I of Handshake Protocol



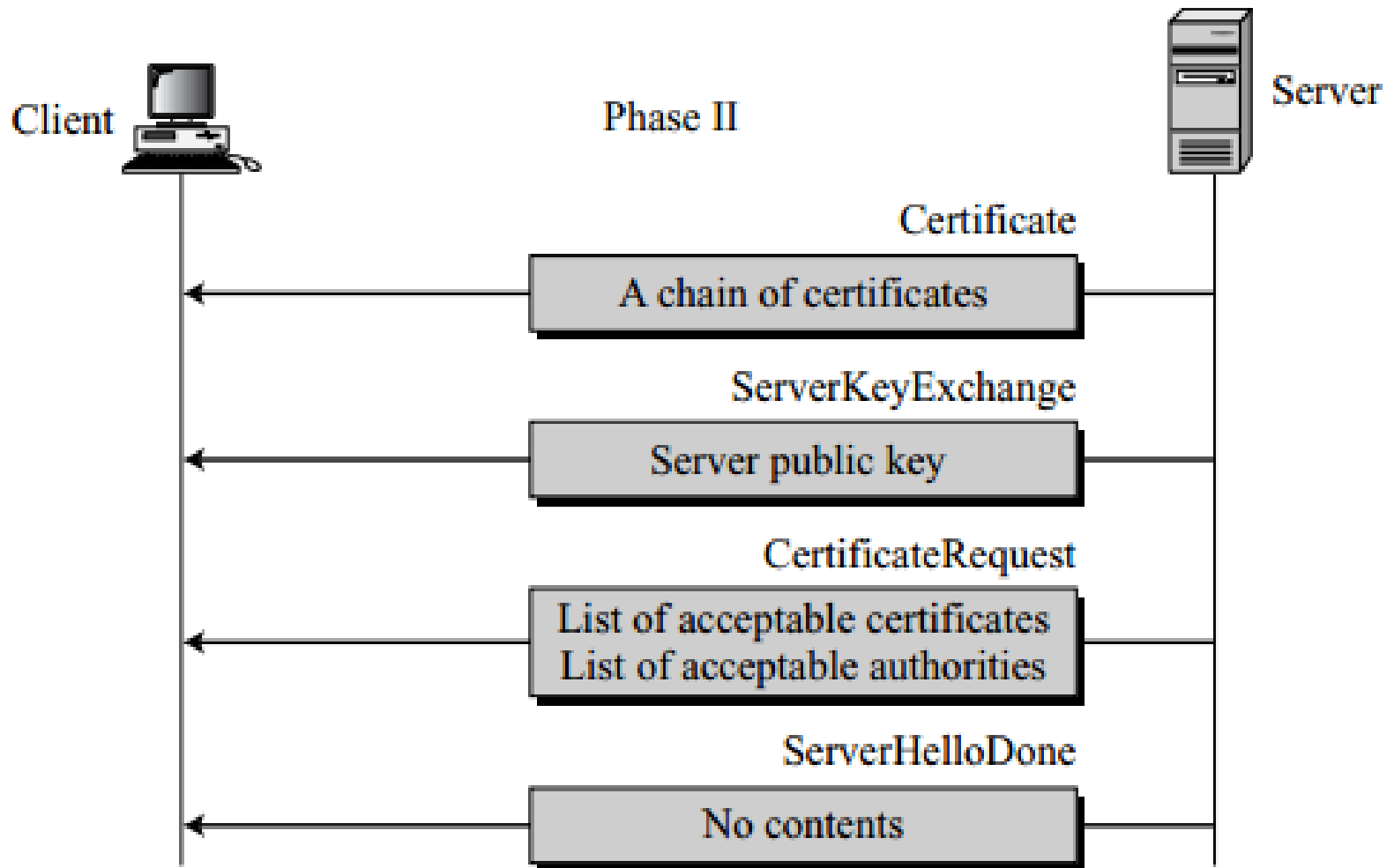
Phase I of Handshake Protocol

- After Phase I, the client and server know the following:
 - The version of SSL
 - The algorithms for key exchange, message authentication, and encryption
 - The compression method
 - The two random numbers for key generation

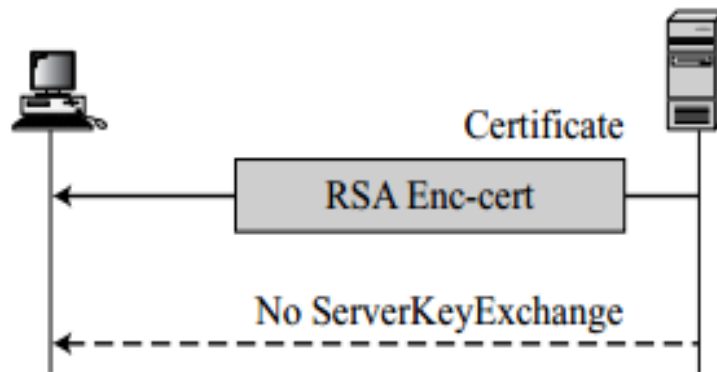
Phase II: Server Key Exchange and Authentication

- The server authenticates itself **if needed**.
 - The server may send its certificate, public key, and may also request certificates from the client.
- After Phase II,
 - The server is authenticated to the client.
 - The client knows the public key of the server if required.

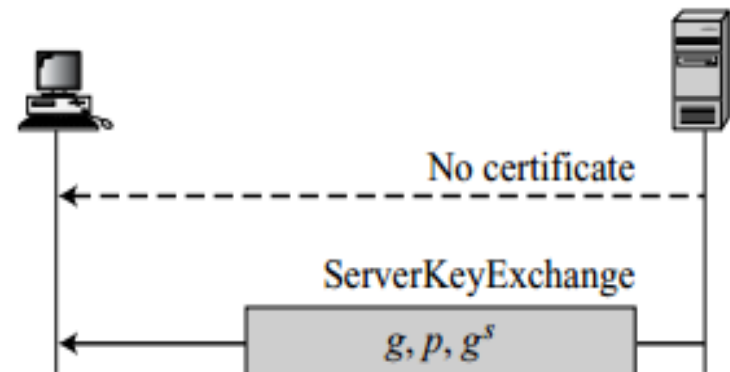
Phase II of Handshake Protocol



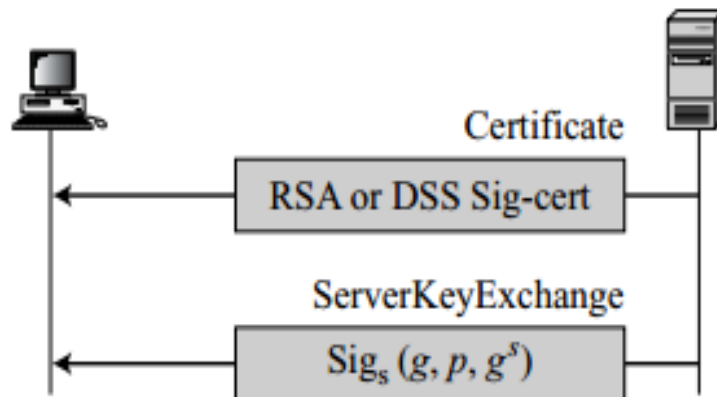
Four cases in Phase II of Handshake Protocol



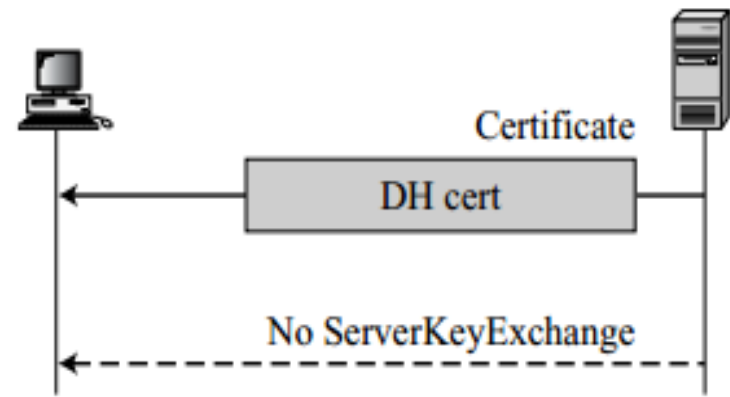
a. RSA



b. Anonymous DH



c. Ephemeral DH



d. Fixed DH

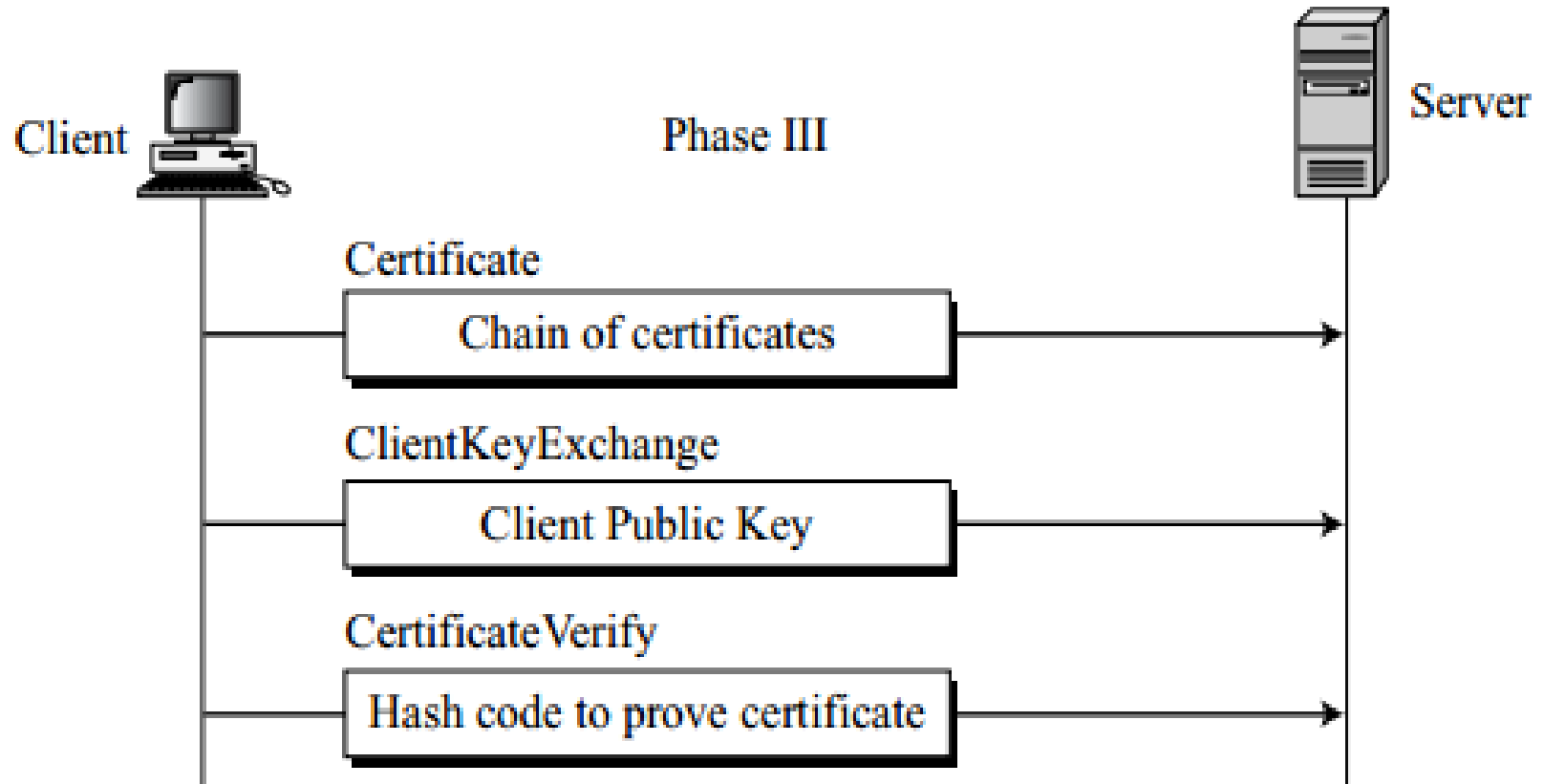
Four cases in Phase II of Handshake Protocol

- **RSA:**
 - the public-key certificate authenticates the server to the client.
- **Anonymous DH:**
 - there is no Certificate message
 - the server is not authenticated
- **Ephemeral DH**
 - server is authenticated to the client (**possession of the private key is proof**)

Four cases in Phase II of Handshake Protocol

- Fixed DH:
 - CA is authenticated to the client
 - CA claims that the half-key belongs to the server

Phase III of Handshake Protocol: Client Key Exchange and Authentication



Phase III: Client Key Exchange and Authentication

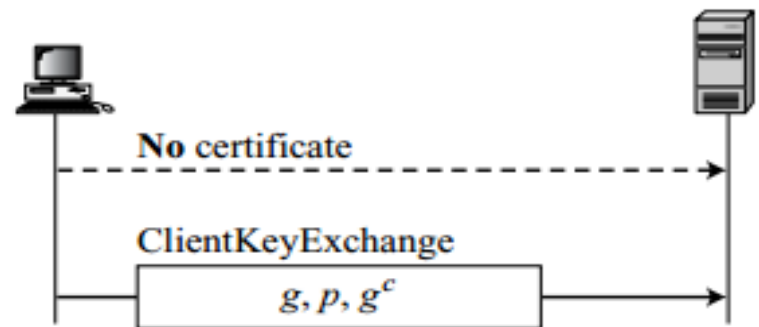
- After Phase III,
 - The client is authenticated for the server.
 - Both the client and the server know the pre-master secret.

Four cases in Phase III of Handshake Protocol

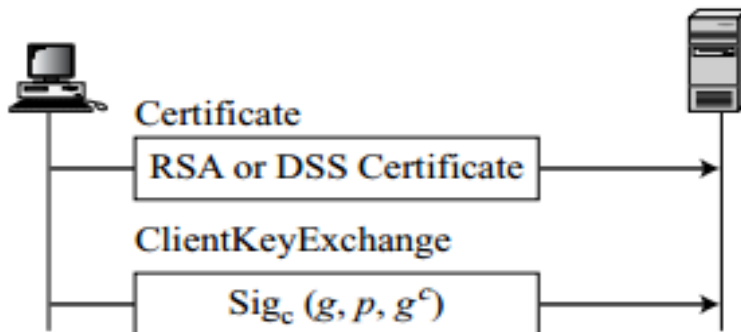
S  Encrypted with server's public key
Sig_c: Signed with client's public key



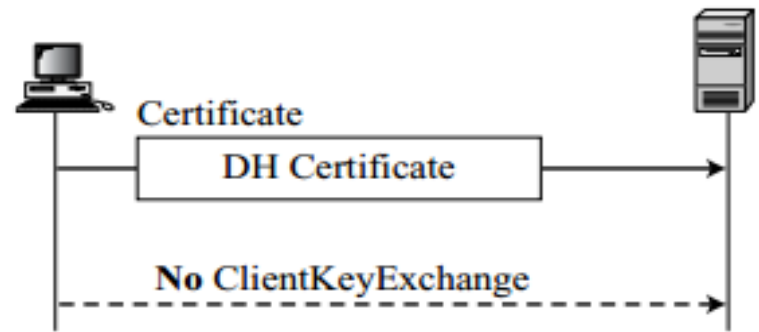
a. RSA



b. Anonymous DH



c. Ephemeral DH



d. Fixed DH

Four cases in Phase III of Handshake Protocol

- **RSA:**
 - There is no Certificate message unless the server has explicitly requested one in Phase II
- **Anonymous DH:**
 - There is no Certificate message
 - client is not authenticated to the server
- **Ephemeral DH:**
 - client usually has a certificate
 - If the client does not have the certificate, and the server asks for it, the client sends an Alert message to warn the client

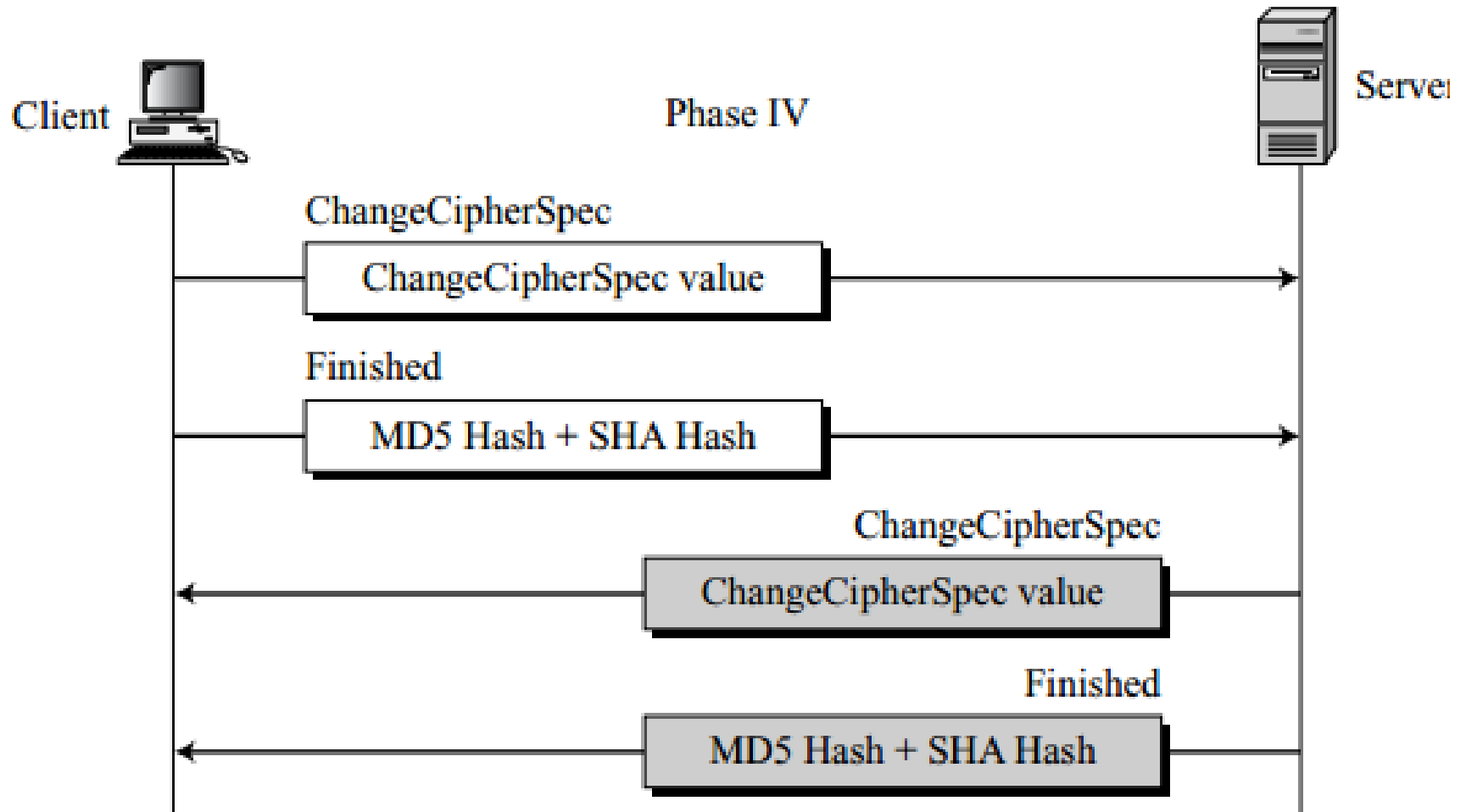
Four cases in Phase III of Handshake Protocol

- Fixed DH
 - The client is authenticated to the server by sending the DH certificate

Phase IV: Finalizing and Finishing

- After Phase IV, the client and server are ready to exchange data
- In Phase IV, the client and server send messages to:
 - change cipher specification
 - finish the handshaking protocol.

Phase IV of Handshake Protocol



ChangeCipherSpec Protocol

- SSL mandates that the parties cannot use the parameters or secrets (done in handshake protocol)
 - until they have sent or received a special message, the **ChangeCipherSpec message**, which is exchanged during the Handshake protocol and defined in the ChangeCipherSpec Protocol.

ChangeCipherSpec Protocol

- The sender and the receiver need two states:
 - **pending state**, keeps track of the parameters and secrets.
 - **active state**, holds parameters and secrets used by the Record Protocol to sign/verify or encrypt/decrypt messages
- Each state holds two sets of values:
 - **Read** (inbound)
 - **write** (outbound).

Alert Protocol

- SSL uses the Alert Protocol for reporting errors and abnormal conditions.
- It has only one message type, the Alert message

Alerts defined for SSL

<i>Value</i>	<i>Description</i>	<i>Meaning</i>
0	<i>CloseNotify</i>	Sender will not send any more messages.
10	<i>UnexpectedMessage</i>	An inappropriate message received.
20	<i>BadRecordMAC</i>	An incorrect MAC received.
30	<i>DecompressionFailure</i>	Unable to decompress appropriately.
40	<i>HandshakeFailure</i>	Sender unable to finalize the handshake.
41	<i>NoCertificate</i>	Client has no certificate to send.
42	<i>BadCertificate</i>	Received certificate corrupted.
43	<i>UnsupportedCertificate</i>	Type of received certificate is not supported.
44	<i>CertificateRevoked</i>	Signer has revoked the certificate.
45	<i>CertificateExpired</i>	Certificate expired.
46	<i>CertificateUnknown</i>	Certificate unknown.
47	<i>IllegalParameter</i>	An out-of-range or inconsistent field.

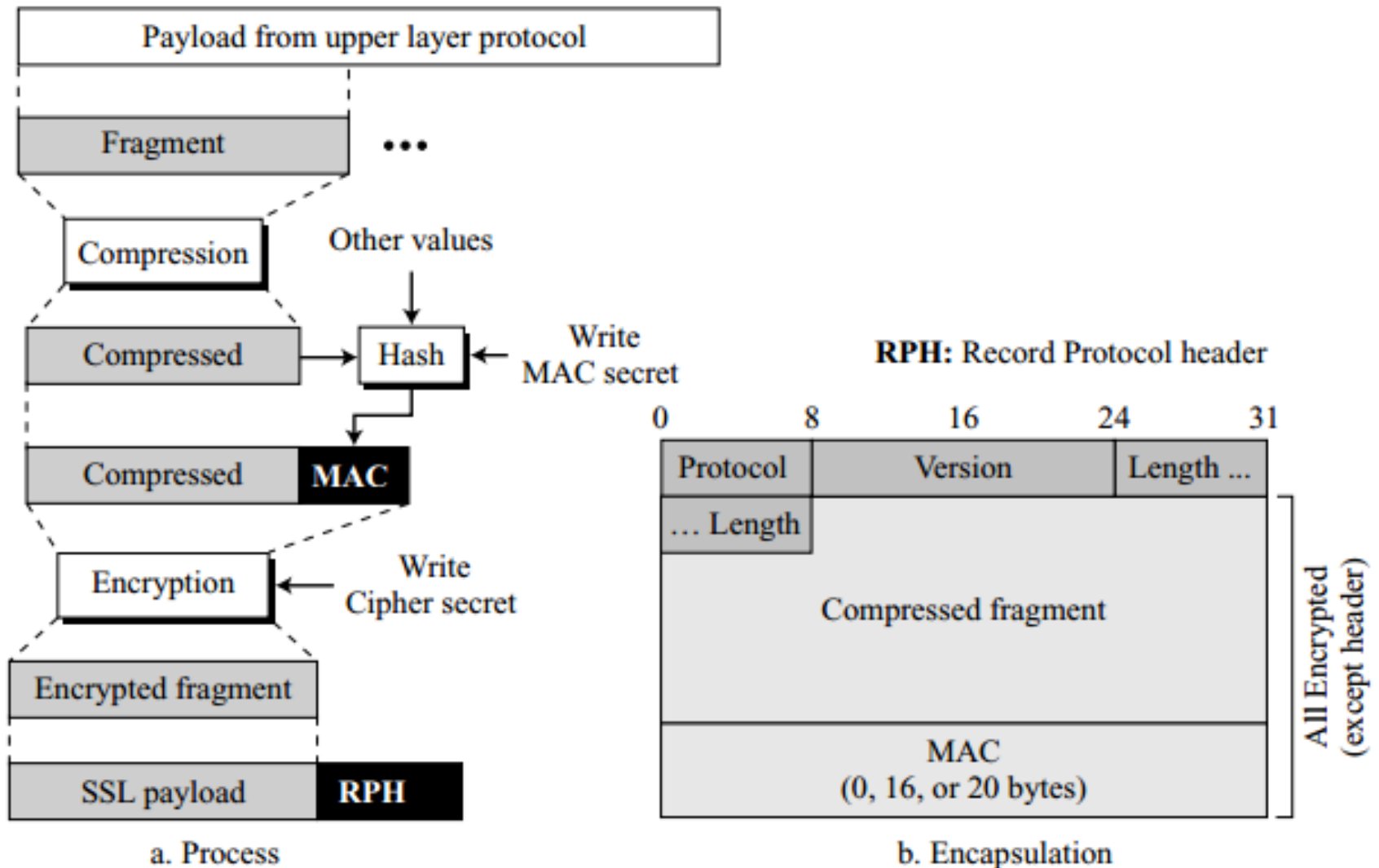
Record Protocol

- It carries messages from the upper layer (Handshake Protocol, ChangeCipherSpec Protocol, Alert Protocol, or application layer).
 - message is fragmented and optionally compressed
 - a MAC is added to the compressed message
 - The compressed fragment and the MAC are encrypted
 - SSL header is added to the encrypted message

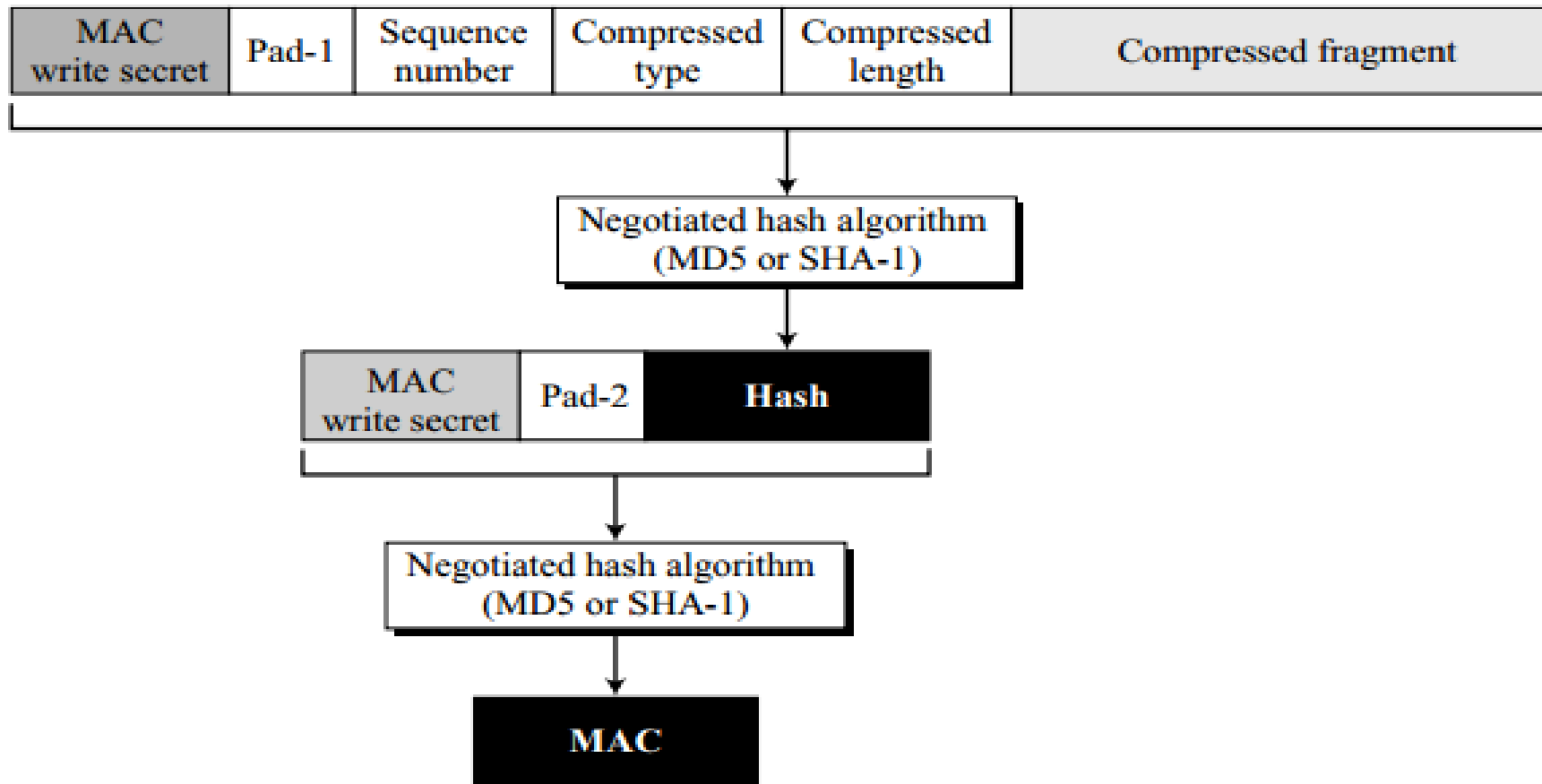
Processing done by the Record Protocol

- Fragmentation/Combination
- Compression/Decompression
- Signing/Verifying
- Encryption/Decryption
- Framing/Deframing

Processing done by the Record Protocol



Calculation of MAC



Pad-1: Byte 0x36 (00110110) repeated 48 times for MD5 and 40 times for SHA-1

Pad-2: Byte 0x5C (01011100) repeated 48 times for MD5 and 40 times for SHA-1

Processing done by the Record Protocol

- At the sender, a message from the application layer is **fragmented** into blocks of 2^{14} bytes
- At the receiver, the fragments are **combined** together to make a replica of the original message.
- The **compression method** needs to be lossless
 - size of the fragment must not exceed 1024 bytes
 - compression/decompression is optional in SSL.

Processing done by the Record Protocol

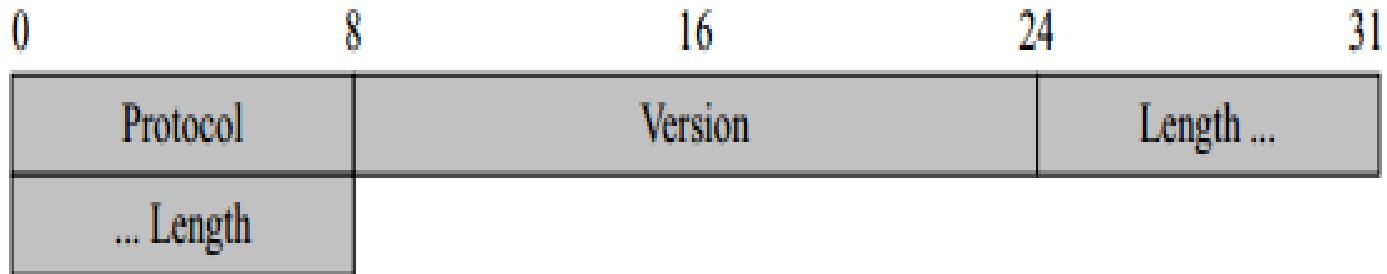
- At the sender, the compressed fragment and the hash are **encrypted** using the cipher write secret.
 - At the receiver, the received message is **decrypted** using the cipher read secret.
- After the encryption, the Record Protocol header is added at the sender (**Framing**).
 - The header is removed at the receiver before decryption (**Deframing**).

SSL Message Formats

- Record Protocol message encapsulates messages from four different sources at the sender site.
- At the receiver site, the Record Protocol decapsulates the messages and delivers them to different destinations.

Record Protocol general header

- Record Protocol has a general header that is added to each message coming from the sources

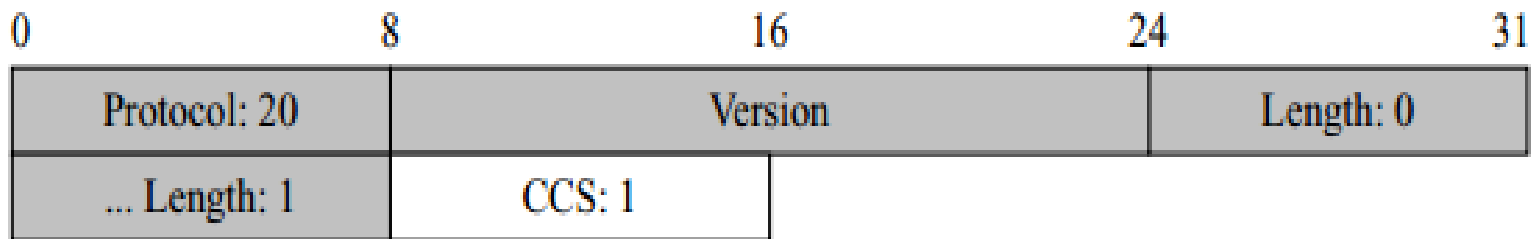


Record Protocol general header

- **Protocol field** defines the source or destination of the encapsulated message.
 - It is used for multiplexing and demultiplexing.
 - The values are:
 - 20 (ChangeCipherSpec Protocol)
 - 21 (Alert Protocol)
 - 22 (Handshake Protocol)
 - 23 (data from the application layer)
- **Version** field defines the version of the SSL; one byte is the major version and the other is the minor. The current version of SSL is 3.0 (major 3 and minor 0).
- **Length** field defines the size of the message (without the header) in bytes.

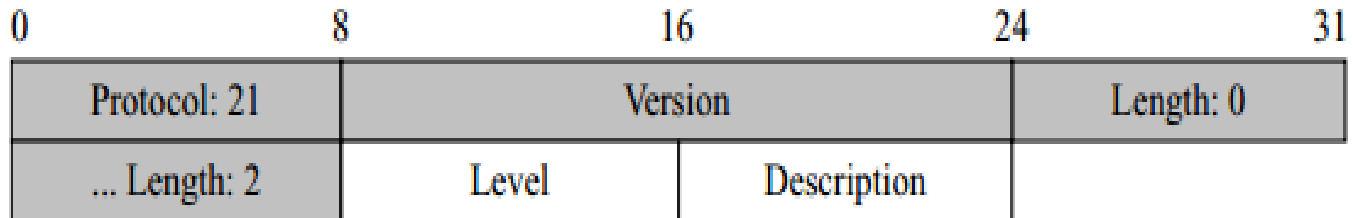
ChangeCipherSpec message

- The message is only one byte, encapsulated in the Record Protocol message with protocol value 20



Alert message

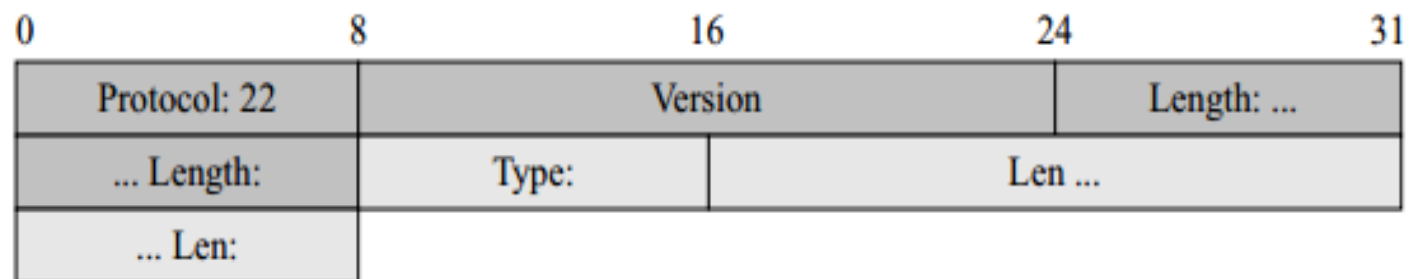
- It has one message that reports errors.
- This single message is encapsulated in the Record Protocol with protocol value 21



Handshake Protocol Messages

- Several messages have been defined for the Handshake Protocol.
 - All have the four-byte **generic header**

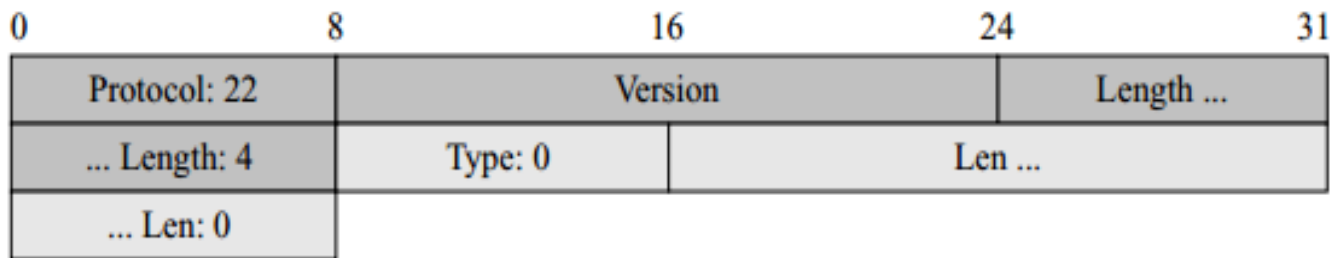
<i>Type</i>	<i>Message</i>
0	HelloRequest
1	ClientHello
2	ServerHello
11	Certificate
12	ServerKeyExchange
13	CertificateRequest
14	ServerHelloDone
15	CertificateVerify
16	ClientKeyExchange
20	Finished



Generic header for Handshake Protocol

HelloRequest Message

- It is a request from the server to the client to restart a session.
- The client then needs to send a ClientHello message and negotiate the security parameters.



HelloRequest message

0	8	16	24	31
Protocol: 22		Version		Length ...
... Length		Type: 1	Len ...	
... Len		Proposed version		
Client random number (32 bytes)				
				ID length
Session ID (variable length)				
Cipher suite length		Cipher suites (variable numbers, each of 2 bytes)		
Com. methods length		Compression methods (variable number, each of 1 byte)		

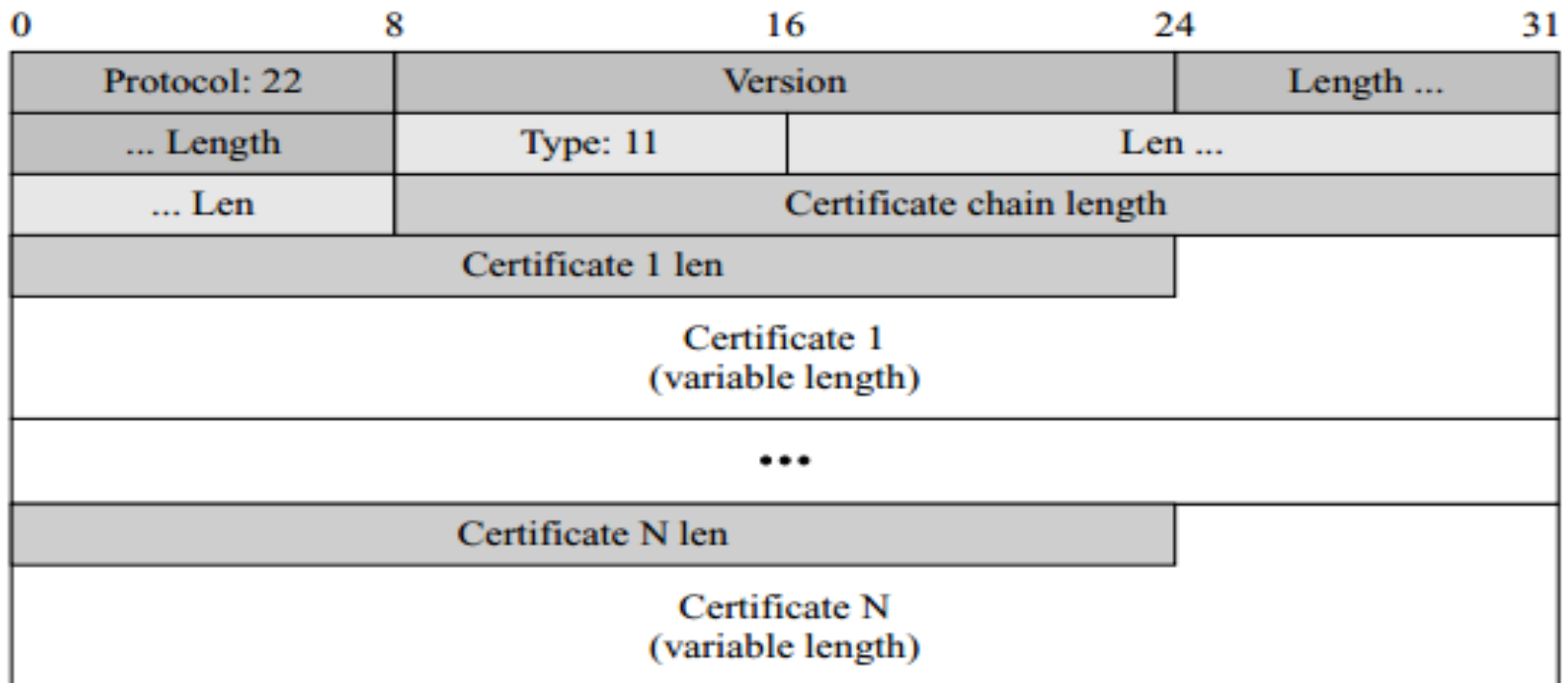
ClientHello message

ServerHello Message

0	8	16	24	31
Protocol: 22		Version		Length ...
... Length		Type: 2	Len ...	
... Len		Proposed version		
Server random number (32 bytes)				ID length
Session ID (variable length)				
Selected cipher suite		Selected com.		

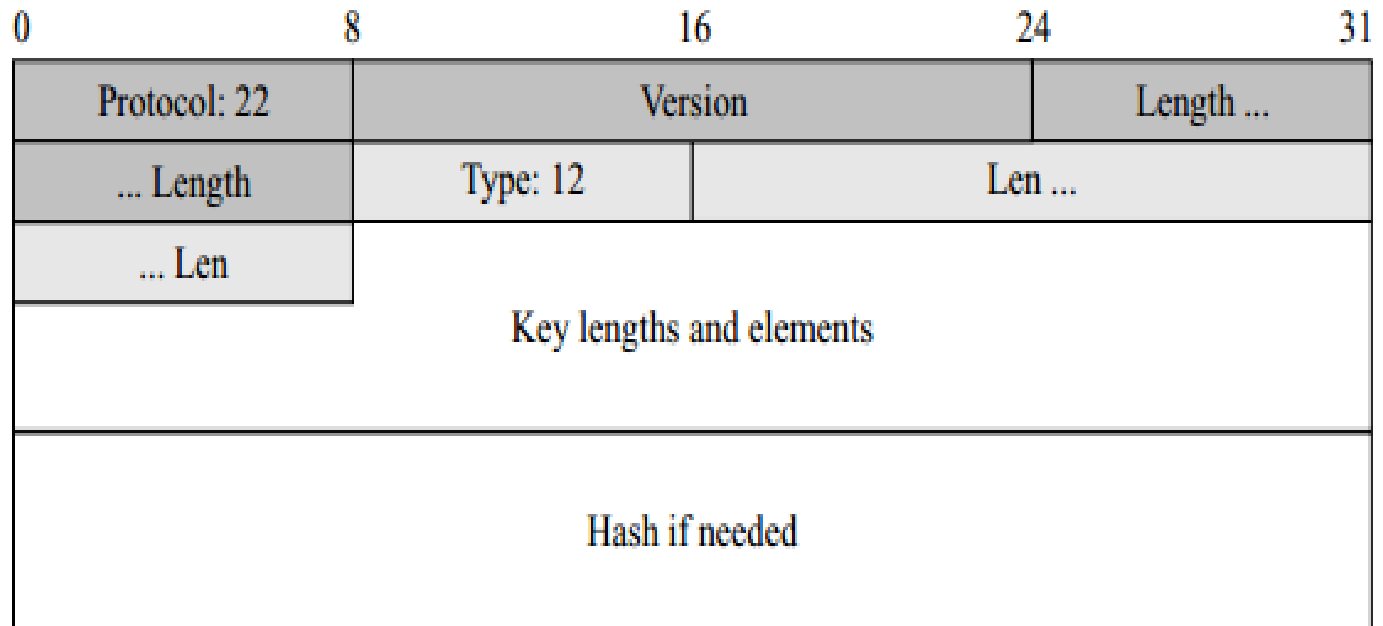
Certificate Message

- The Certificate message can be sent by the client or the server to list the chain of public key certificates



ServerKeyExchange Message

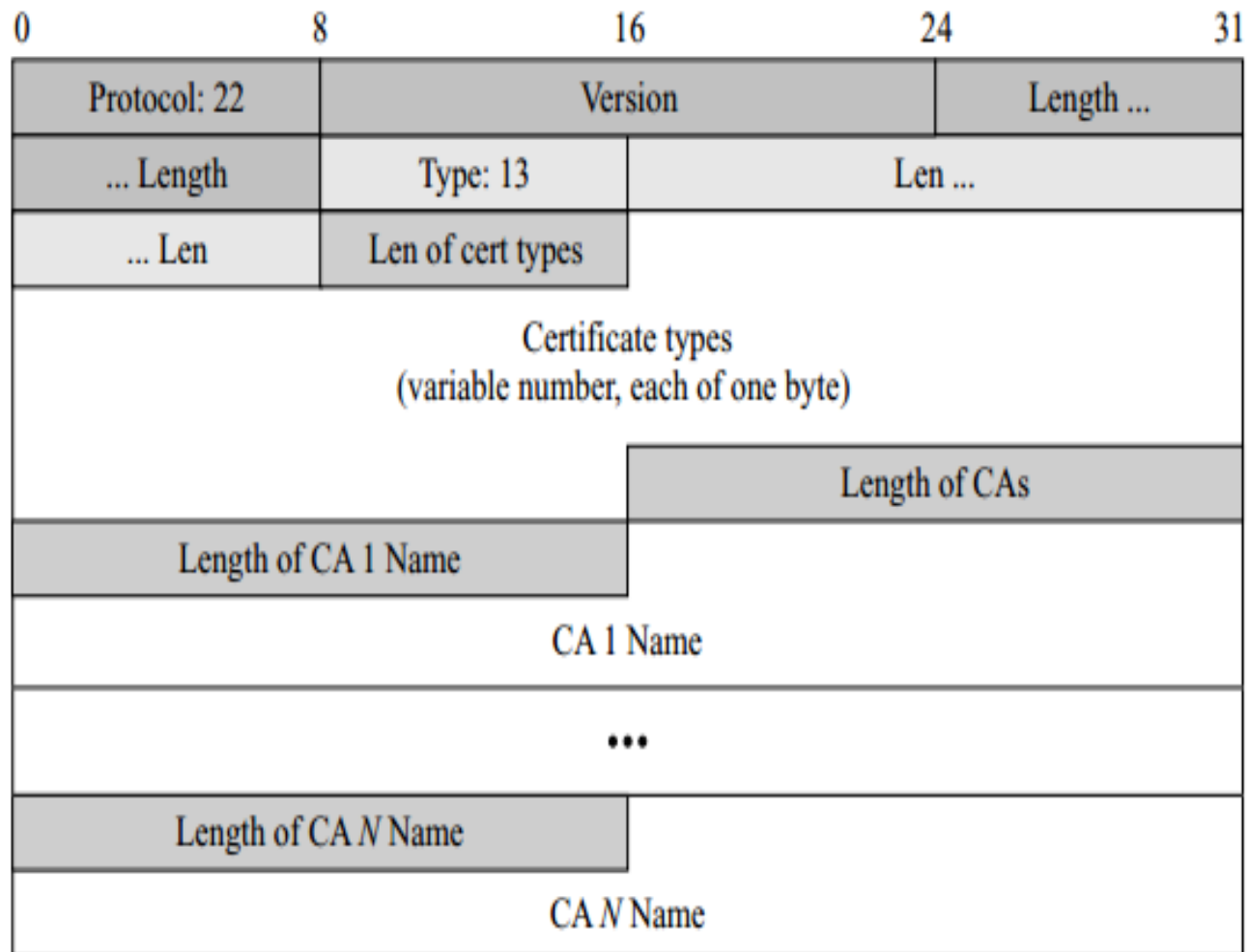
- The message contains the keys generated by the server.
- The format of the message is dependent on the cipher suite selected in the previous message



CertificateRequest Message

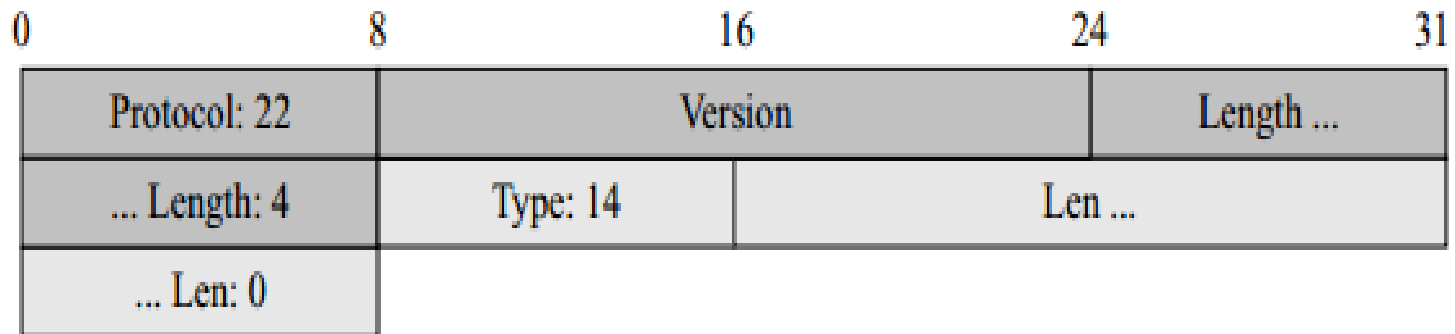
- The message asks the client to authenticate itself to the server using one of the acceptable certificates and one of the certificate authorities named in the message.

CertificateRequest Message



ServerHelloDone Message

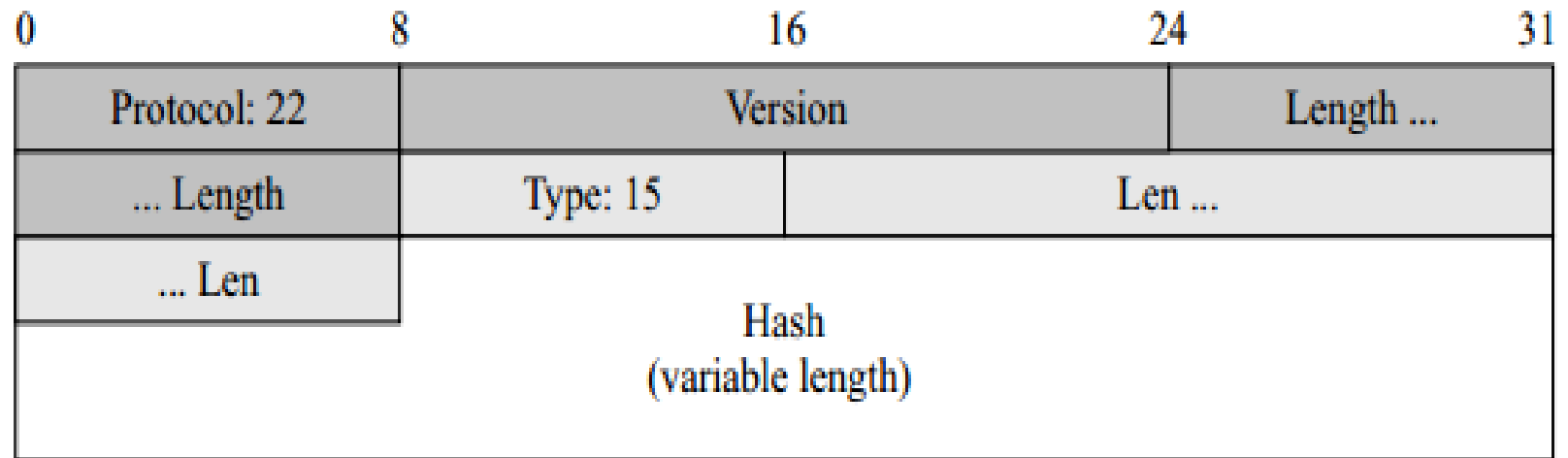
- This message signals that phase II does not carry any extra information



CertificateVerify Message

- It is the last message of Phase III.
- The client proves that it actually owns the private key related to its public-key certificate.
 - client creates a hash of all handshake messages sent before this message and signs them using the MD5 or SHA-1

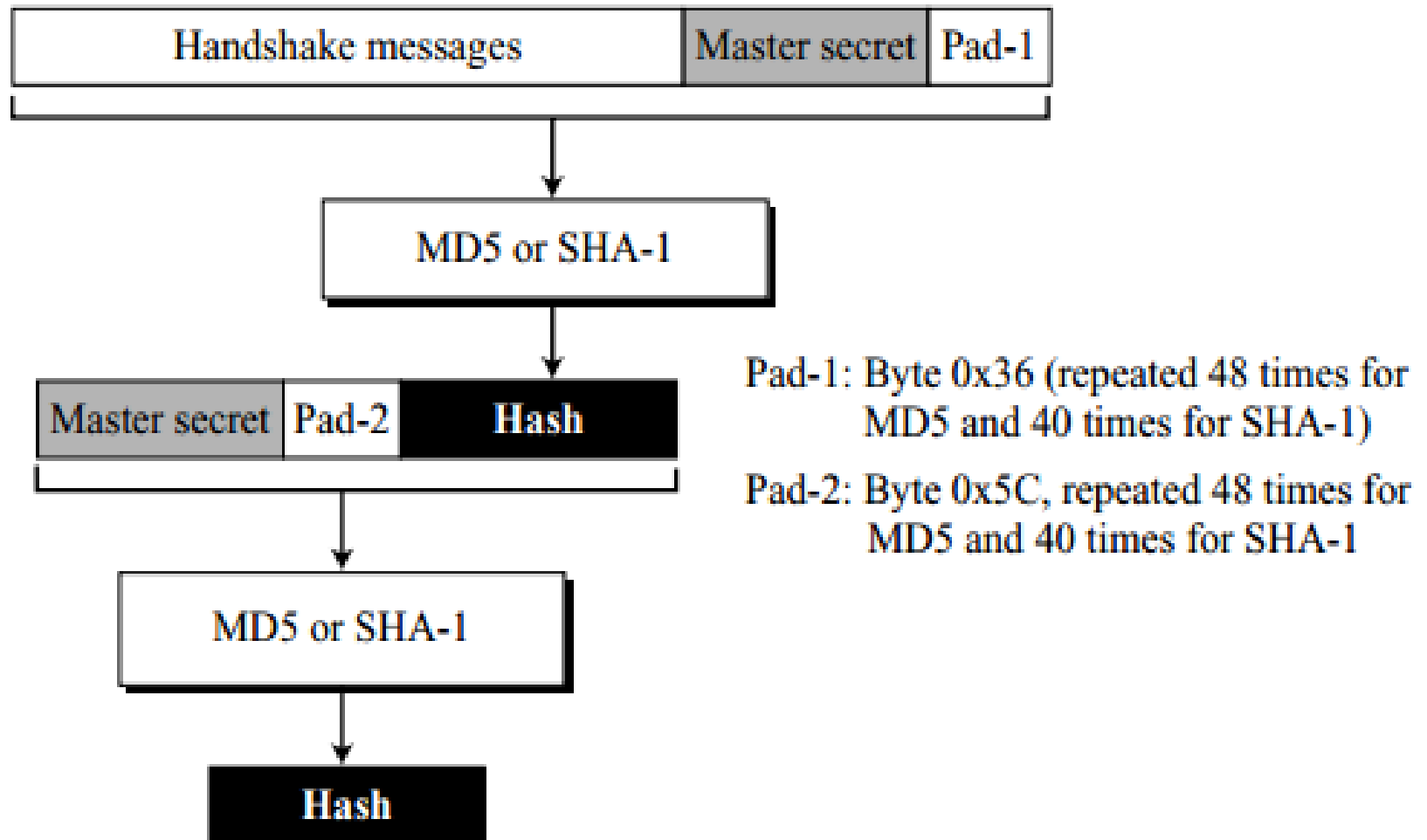
CertificateVerify Message



CertificateVerify Message

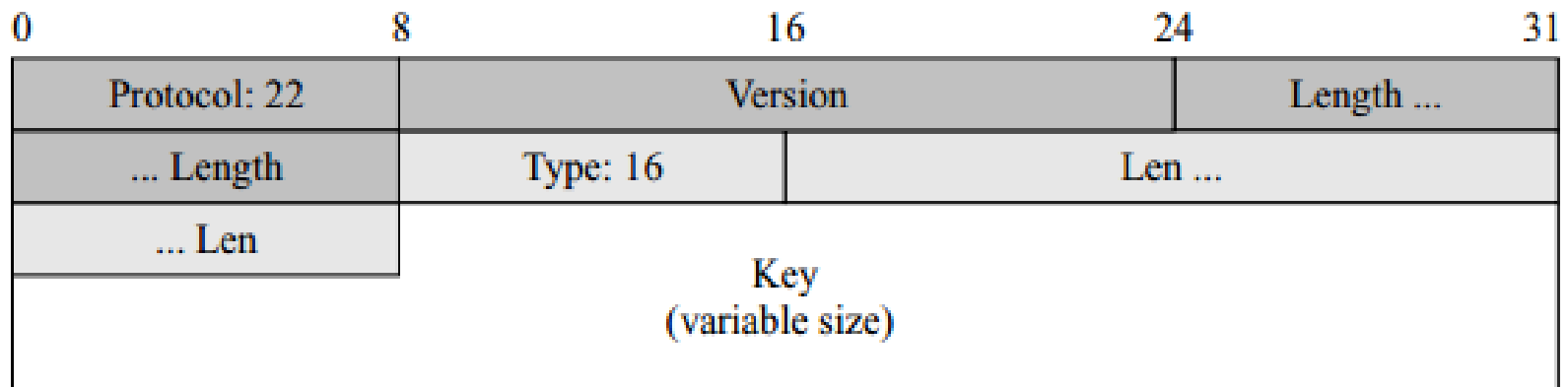
- If the client private key is related to a DSS certificate, length of the hash is 20 bytes.
- If the client private key is related to a RSA certificate, then there are two hashes (concatenated), one based on MD5 and the other based on SHA-1.

Hash calculation for CertificateVerify message



ClientKeyExchange Message

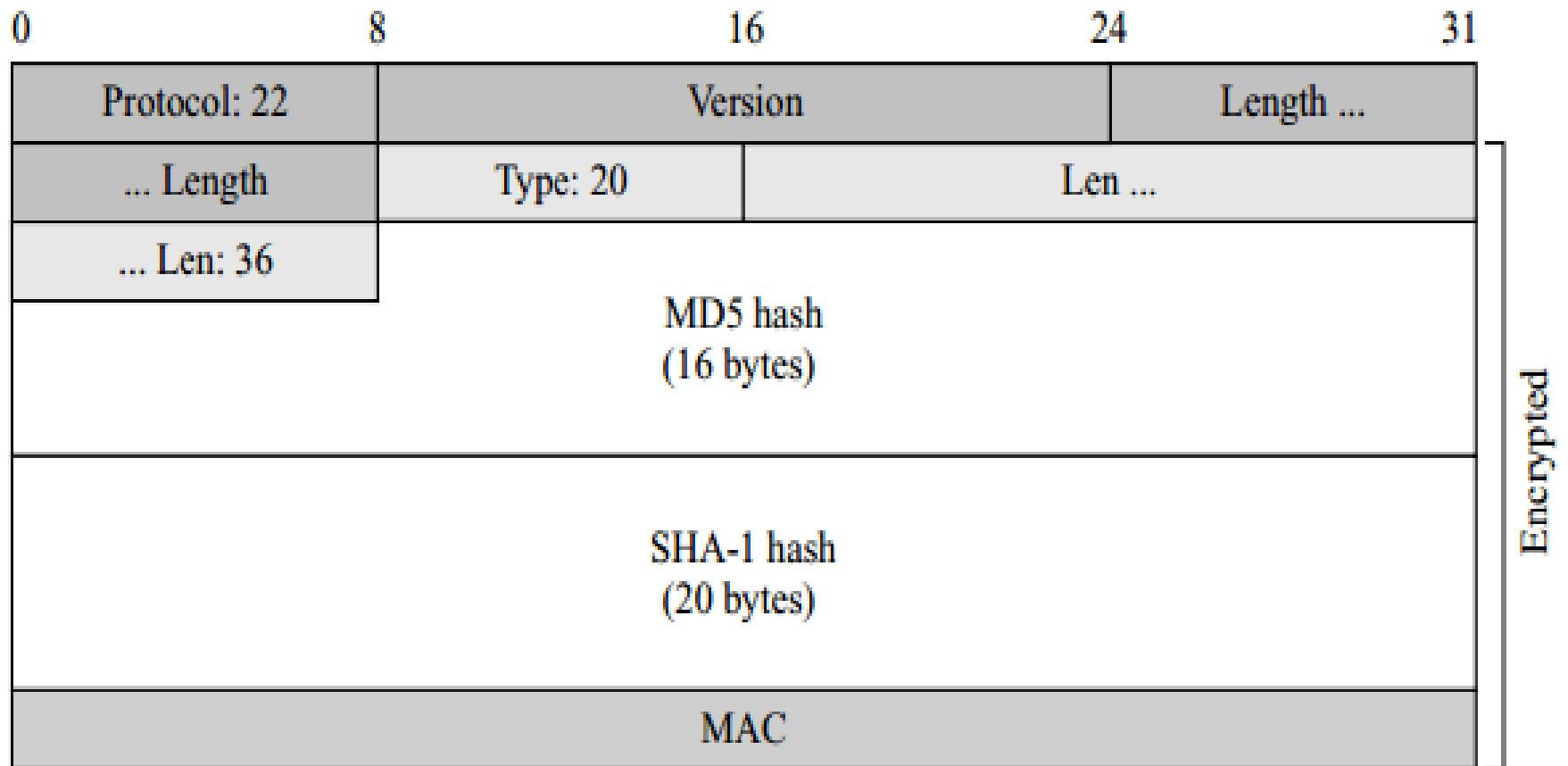
- In this message, the client provides the keys.
- The format of the message depends on the specific key exchange algorithms selected by two parties



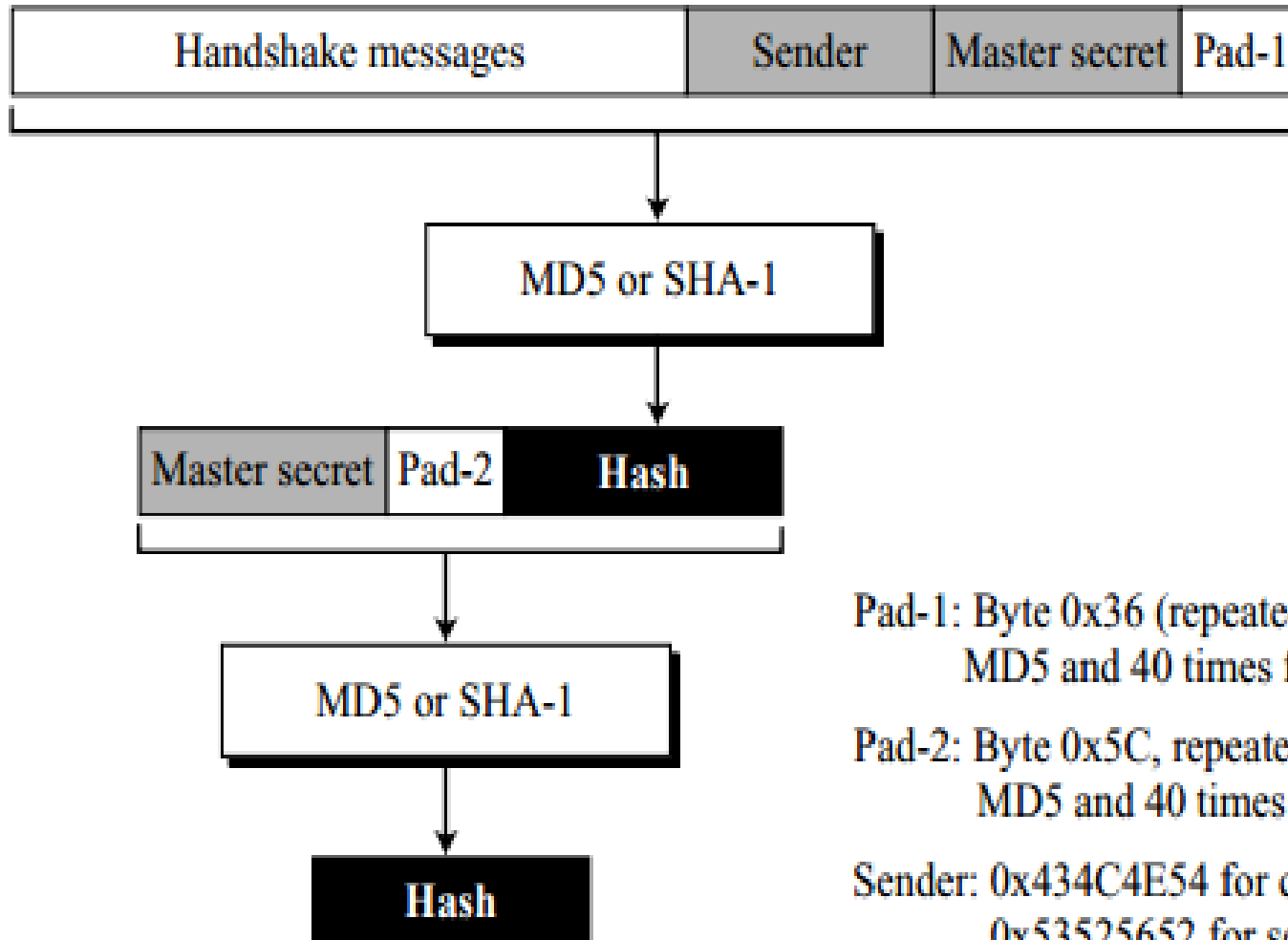
Finished Message

- The Finished message shows that the negotiation is over.
- It contains all of the messages exchanged during handshaking, followed by the sender role, the master secret, and the padding.

Finished Message



Hash calculation for Finished message



Pad-1: Byte 0x36 (repeated 48 times for MD5 and 40 times for SHA-1)

Pad-2: Byte 0x5C, repeated 48 times for MD5 and 40 times for SHA-1

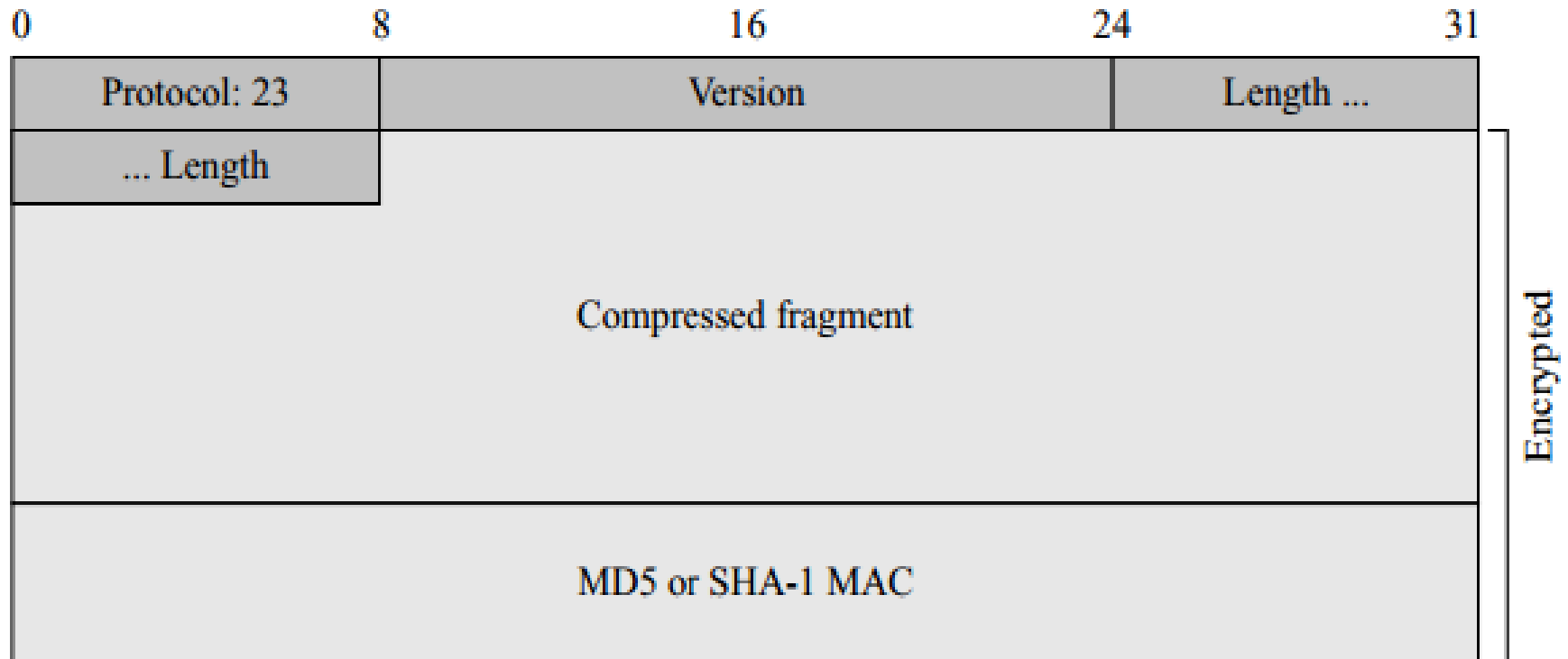
Sender: 0x434C4E54 for client;
0x53525652 for server

Transport Layer Security (TLS)

Transport Layer Security (TLS)

- It is the IETF standard version of the SSL protocol.
- The two are very similar, with slight differences.

TLS Record Protocol message for application data



TLS Record Protocol message for application data

- Version number (major and minor)
 - The current version of SSL is 3.0; the current version of TLS is 1.0.
- Cipher Suite
 - TLS does not support Fortezza for key exchange or for encryption/decryption.
- Generation of Cryptographic Secrets
 - more complex in TLS than in SSL

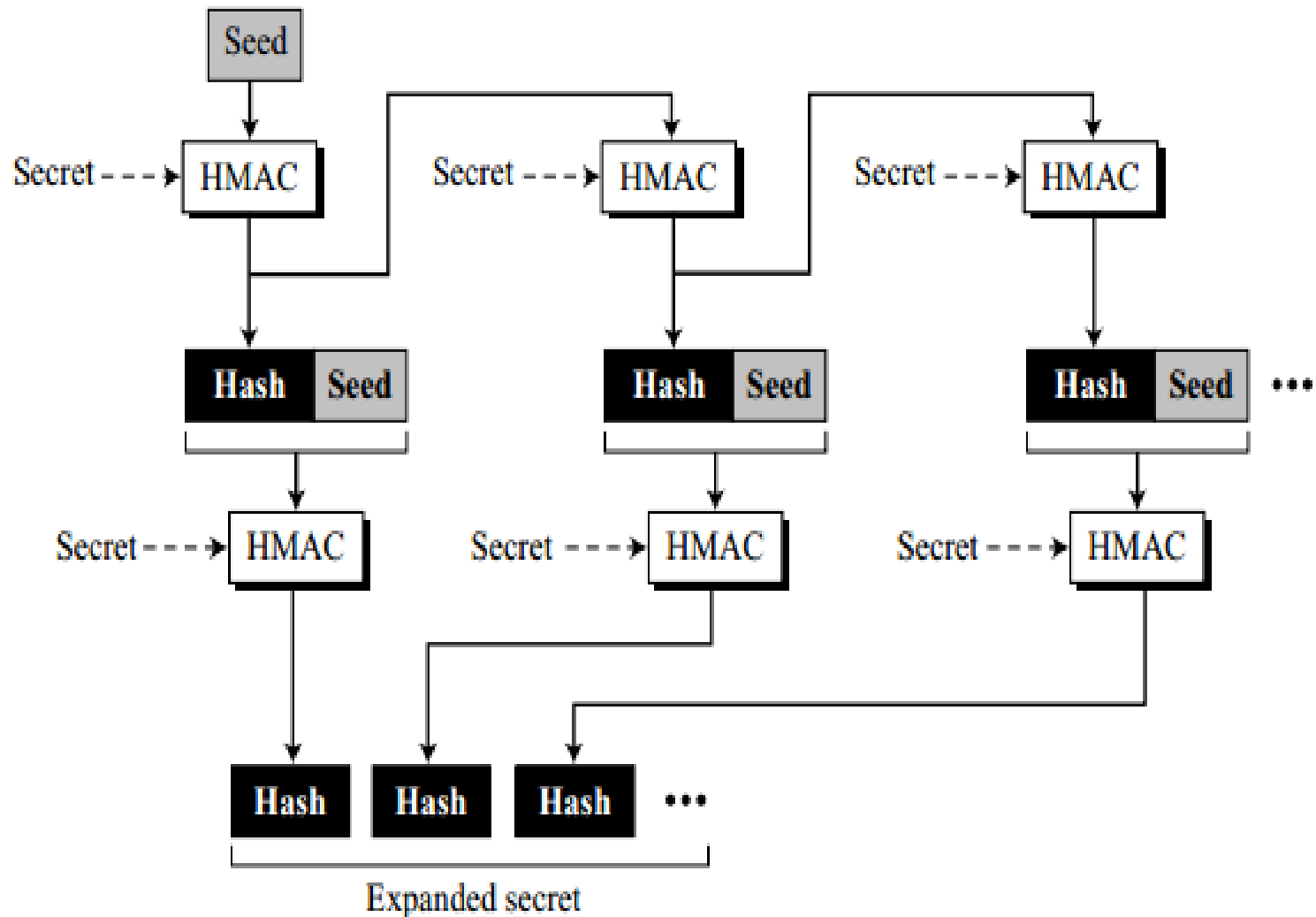
Cipher Suite for TLS

<i>Cipher suite</i>	<i>Key Exchange</i>	<i>Encryption</i>	<i>Hash</i>
TLS_NULL_WITH_NULL_NULL	NULL	NULL	NULL
TLS_RSA_WITH_NULL_MD5	RSA	NULL	MD5
TLS_RSA_WITH_NULL_SHA	RSA	NULL	SHA-1
TLS_RSA_WITH_RC4_128_MD5	RSA	RC4	MD5
TLS_RSA_WITH_RC4_128_SHA	RSA	RC4	SHA-1
TLS_RSA_WITH_IDEA_CBC_SHA	RSA	IDEA	SHA-1
TLS_RSA_WITH_DES_CBC_SHA	RSA	DES	SHA-1
TLS_RSA_WITH_3DES_EDE_CBC_SHA	RSA	3DES	SHA-1
TLS_DH_anon_WITH_RC4_128_MD5	DH_anon	RC4	MD5
TLS_DH_anon_WITH_DES_CBC_SHA	DH_anon	DES	SHA-1
TLS_DH_anon_WITH_3DES_EDE_CBC_SHA	DH_anon	3DES	SHA-1
TLS_DHE_RSA_WITH_DES_CBC_SHA	DHE_RSA	DES	SHA-1
TLS_DHE_RSA_WITH_3DES_EDE_CBC_SHA	DHE_RSA	3DES	SHA-1
TLS_DHE_DSS_WITH_DES_CBC_SHA	DHE_DSS	DES	SHA-1
TLS_DHE_DSS_WITH_3DES_EDE_CBC_SHA	DHE_DSS	3DES	SHA-1
TLS_DH_RSA_WITH_DES_CBC_SHA	DH_RSA	DES	SHA-1
TLS_DH_RSA_WITH_3DES_EDE_CBC_SHA	DH_RSA	3DES	SHA-1
TLS_DH_DSS_WITH_DES_CBC_SHA	DH_DSS	DES	SHA-1
TLS_DH_DSS_WITH_3DES_EDE_CBC_SHA	DH_DSS	3DES	SHA-1

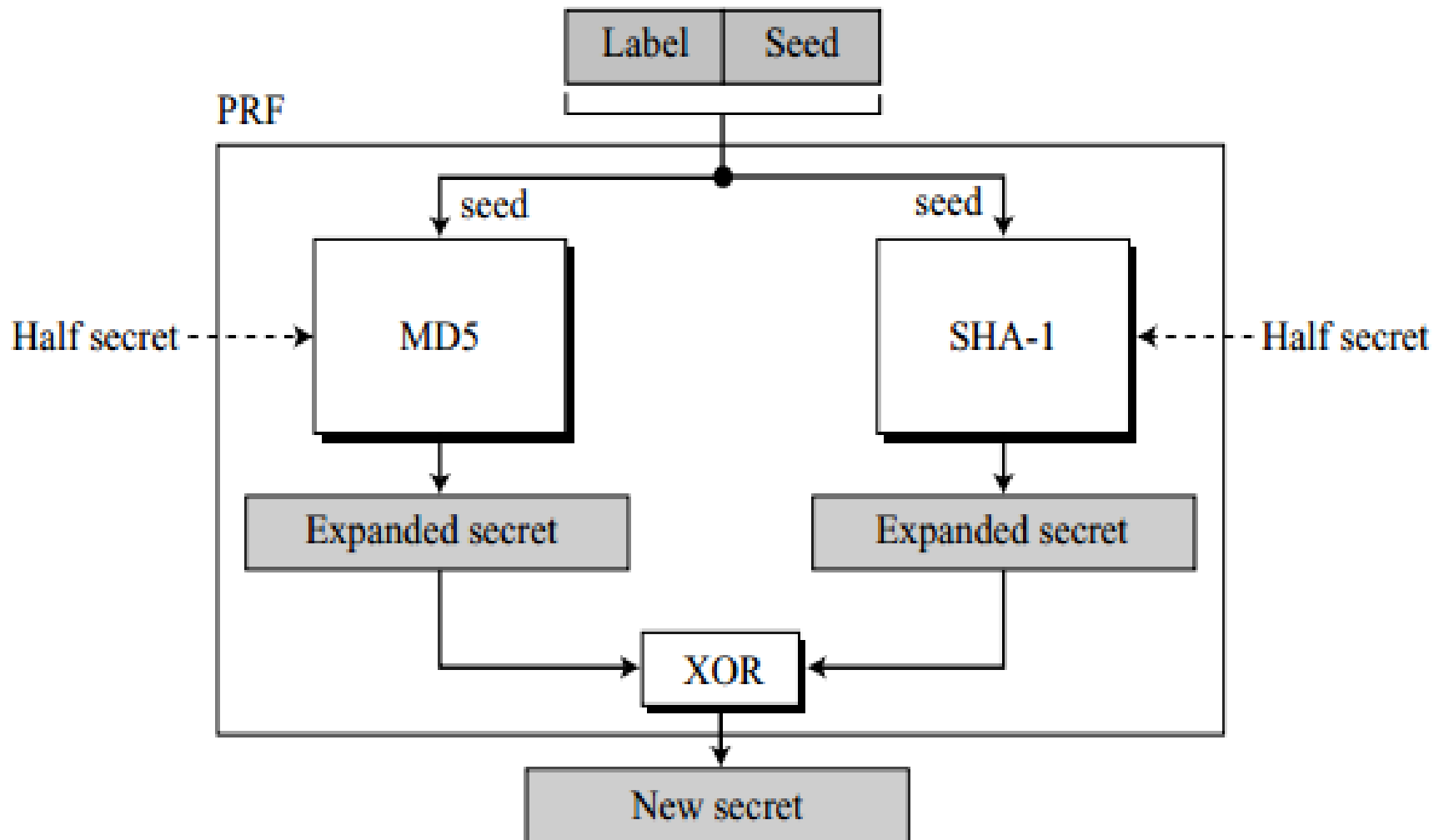
Generation of Cryptographic Secrets in TLS

- TLS first defines two functions:
 - data-expansion function
 - pseudorandom function

Data-expansion function



Pseudorandom Function (PRF)

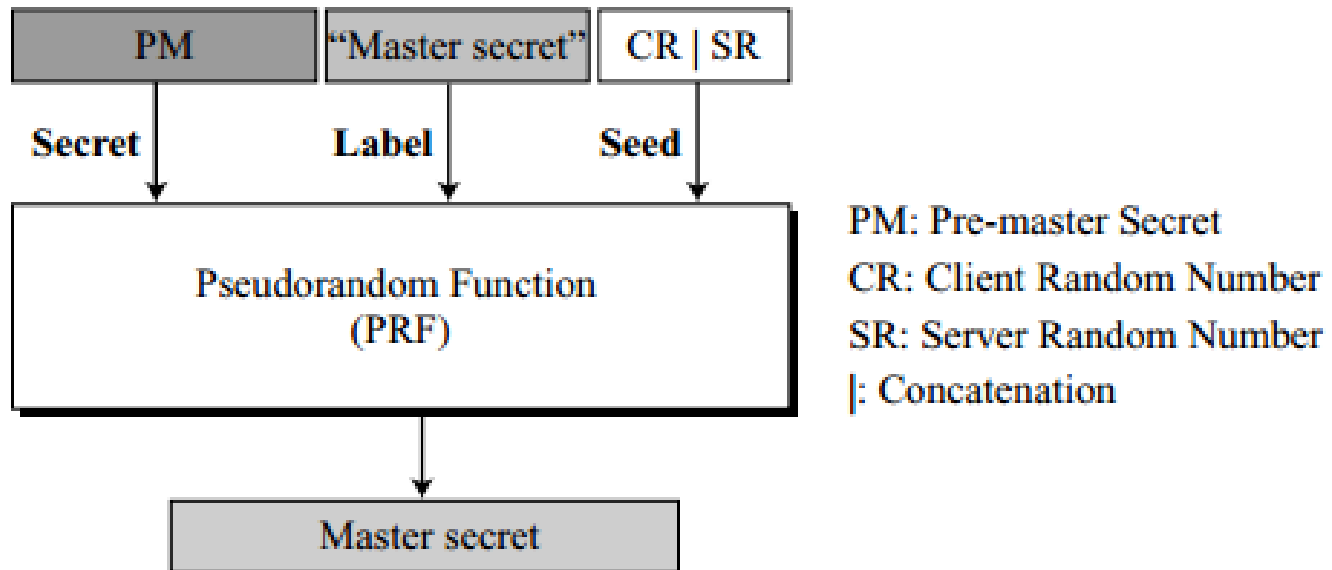


Pre-master Secret in TLS

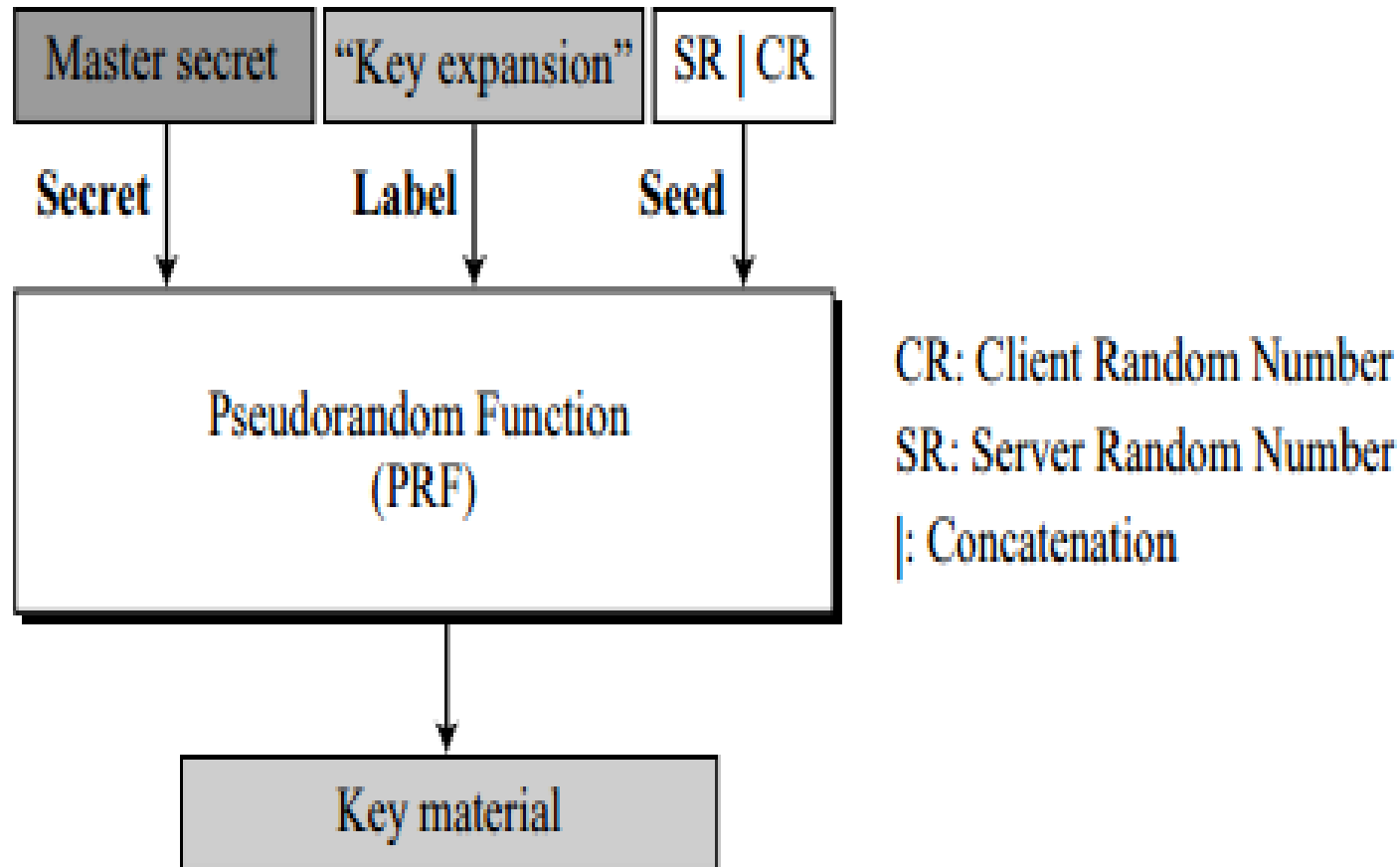
- The generation of the pre-master secret in TLS is exactly the same as in SSL.

Master secret generation in TLS

- TLS uses the PRF function to create the master secret from the pre-master secret.



Key material generation in TLS



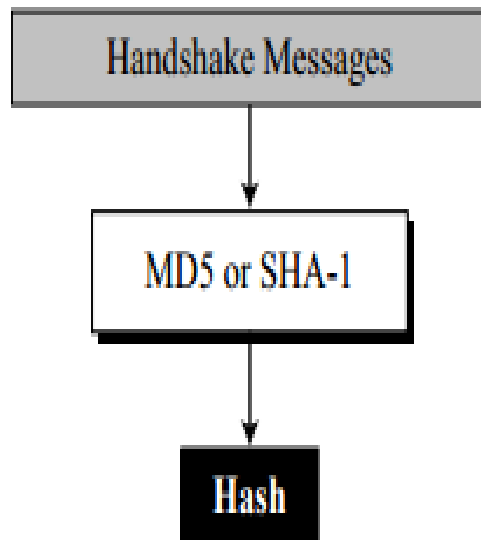
Alerts defined for TLS

<i>Value</i>	<i>Description</i>	<i>Meaning</i>
0	<i>CloseNotify</i>	Sender will not send any more messages.
10	<i>UnexpectedMessage</i>	An inappropriate message received.
20	<i>BadRecordMAC</i>	An incorrect MAC received.
21	<i>DecryptionFailed</i>	Decrypted message is invalid.
22	<i>RecordOverflow</i>	Message size is more than $2^{14} + 2048$.
30	<i>DecompressionFailure</i>	Unable to decompress appropriately.
40	<i>HandshakeFailure</i>	Sender unable to finalize the handshake.
42	<i>BadCertificate</i>	Received certificate corrupted.
43	<i>UnsupportedCertificate</i>	Type of received certificate is not supported.
44	<i>CertificateRevoked</i>	Signer has revoked the certificate.
45	<i>CertificateExpired</i>	Certificate has expired.
46	<i>CertificateUnknown</i>	Certificate unknown.
47	<i>IllegalParameter</i>	A field out of range or inconsistent with others.
48	<i>UnknownCA</i>	CA could not be identified.

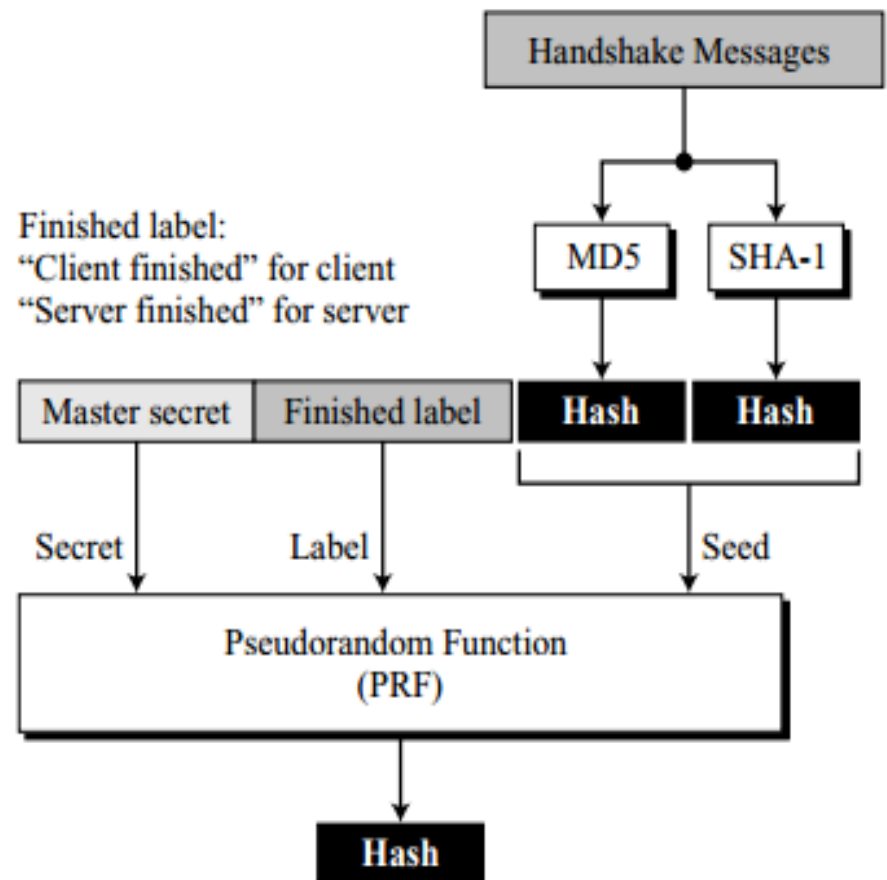
Handshake Protocol for TLS

- TLS has made some changes in
 - CertificateVerify message
 - The hash in the TLS is only over the handshake messages
 - Finished message

Hash for CertificateVerify message in TLS



Hash for Finished message in TLS



Record Protocol for TLS

- TLS uses HMAC for signing the message

